Memory Management

Memory management

- We have seen how CPU can be shared by a set of processes
  - Improve system performance
  - Process management
- Need to keep several process in memory
  - Share memory
- Learn various techniques to manage memory
  - Hardware dependent

Memory management

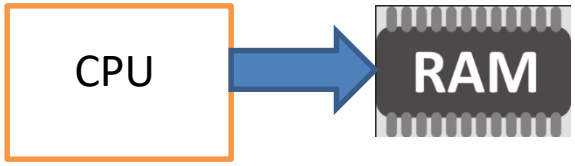
What are we going to learn?

- ***Basic Memory Management:*** logical vs. physical address space, protection, contiguous memory allocation, paging, segmentation, segmentation with paging.

- ***Virtual Memory:*** background, demand paging, performance, page replacement, page replacement algorithms (FCFS, LRU), allocation of frames, thrashing.

Background

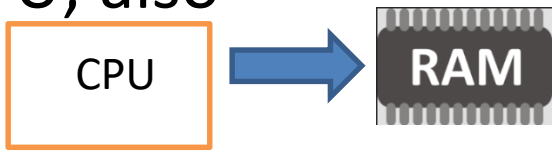
- Program must be brought (from disk) into memory
- Fetch-decode-execute cycle



- Memory unit only sees a stream of addresses + read requests, or address + data and write requests
- Sequence of memory addresses generated by running program

Logical vs. Physical Address Space

Logical address – generated by the CPU; also referred to as **virtual address**



Physical address – address seen by the memory unit

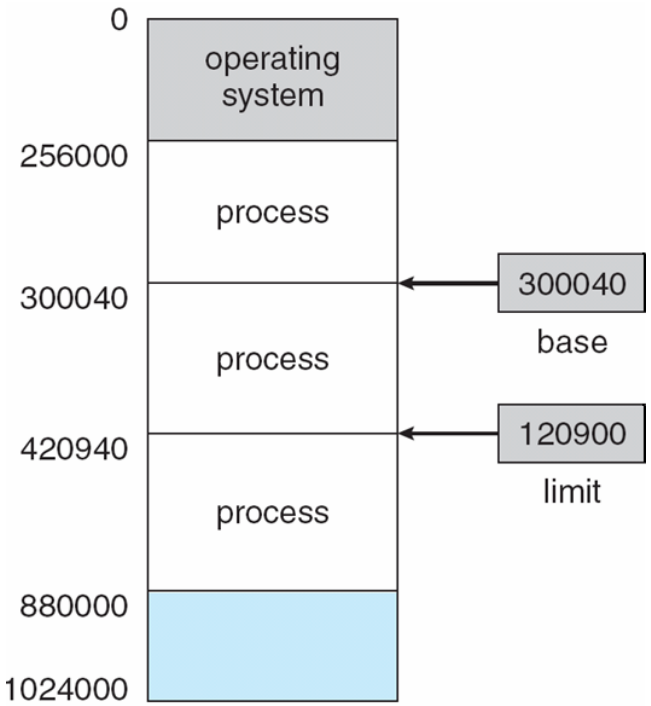
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program

Background

Multiple processes resides in memory

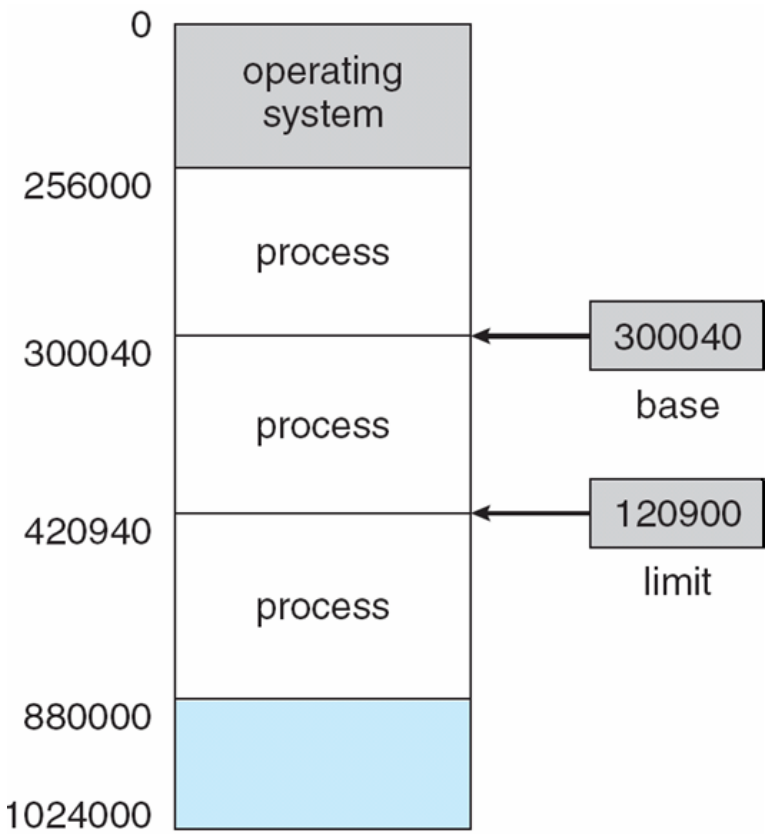
- Protection of memory required to ensure correct operation

1. Protect OS
2. Protect user processes

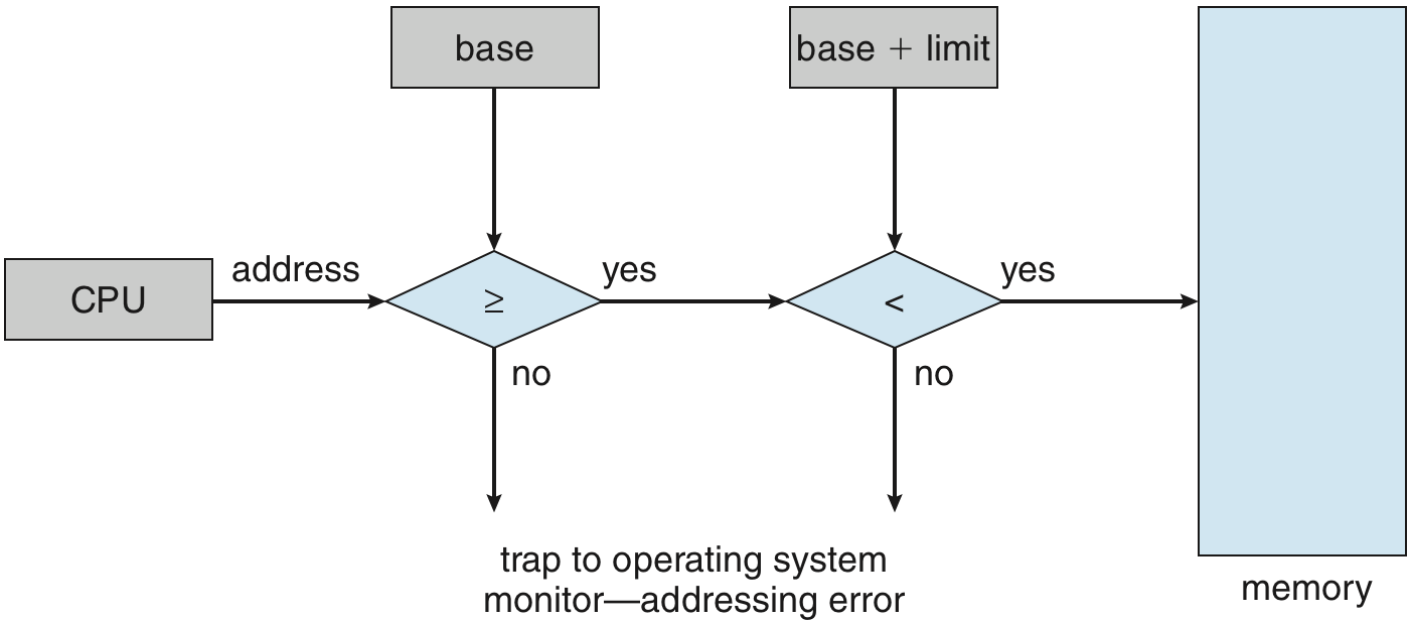


Base and Limit Registers

- A pair of **base** and **limit** registers define the logical address space



Hardware Address Protection with Base and Limit Registers

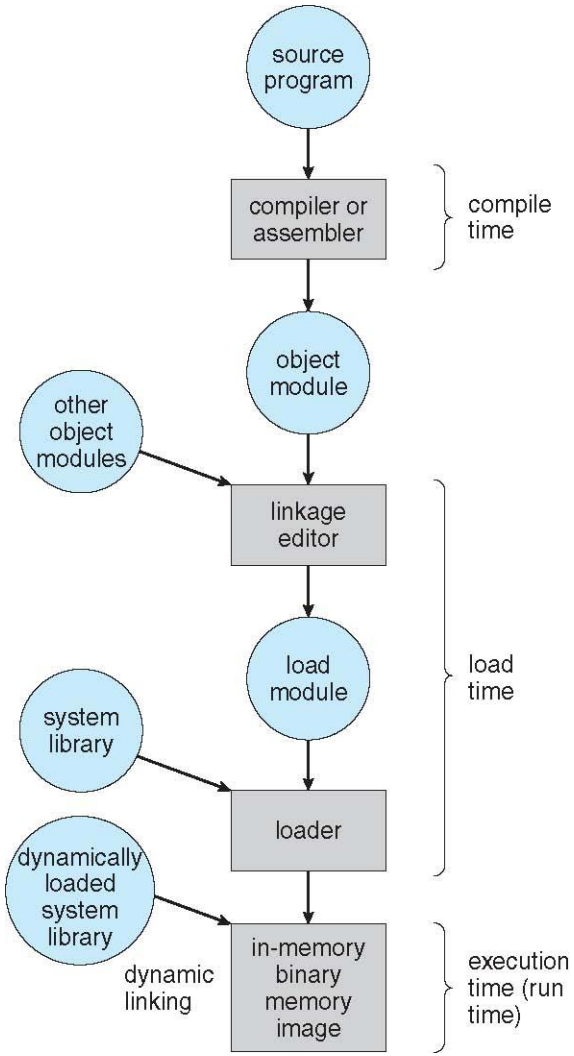


- OS loads the base & limit reg.
- Privileged instruction

Address Binding

- Process resides in main memory
- Associate each data element with memory address
- Further, addresses represented in different ways at different stages of a program's life
  - Source code addresses usually symbolic
  - Compiled code addresses **bind** to relocatable addresses
    - i.e. "14 bytes from beginning of this module"
  - Linker or loader will bind relocatable addresses to absolute addresses
    - i.e. 74014

Multistep Processing of a User Program

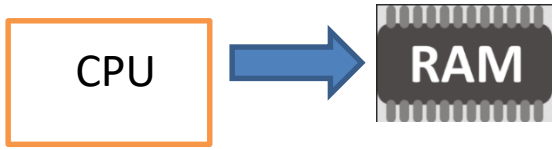


Binding of Instructions and Data to Memory

- Address binding of instructions and data to memory addresses can happen at three different stages
  - **Compile time**: If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
  - **Load time**: Must generate **relocatable code** if memory location is not known at compile time
  - **Execution time**: If the process can be moved during its execution from one memory segment to another
    - Binding delayed until run time
    - Need hardware support for address maps (e.g., base and limit registers)

Logical vs. Physical Address Space

Logical address – generated by the CPU; also referred to as **virtual address**



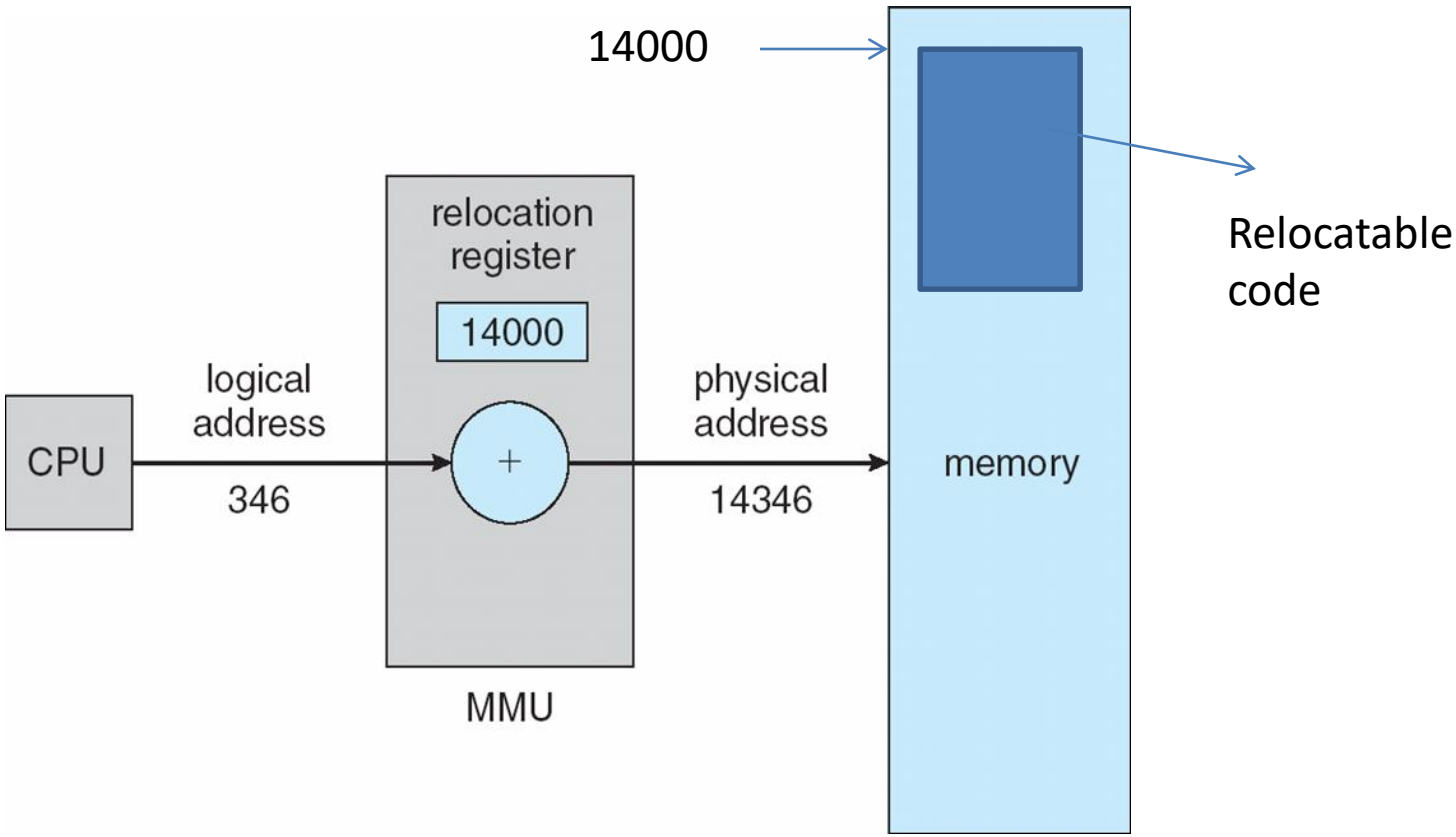
Physical address – address seen by the memory unit

- Logical and physical addresses are the same in compile-time and load-time address-binding schemes;
- logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program

Memory-Management Unit (MMU)

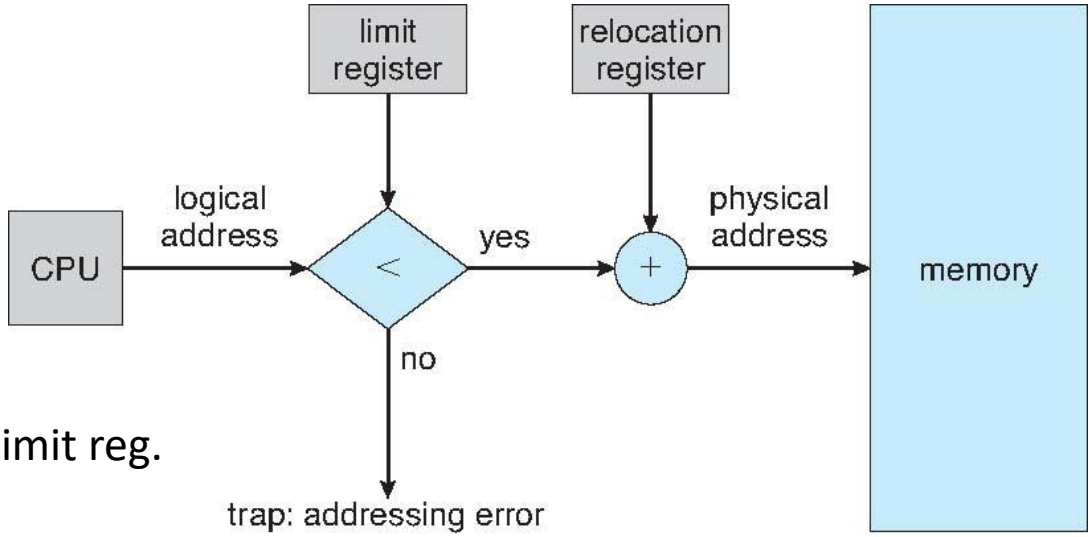
- Hardware device that at run time maps virtual to physical address
- Many methods possible
- To start, consider simple scheme where the value in the **relocation register** is added to every address generated by a user process at the time it is sent to memory
  - **relocation register**
  - MS-DOS on Intel 80x86 used 4 relocation registers
- The user program deals with *logical* addresses (0 to max); it never sees the *real* physical addresses (R to R+max)
  - Say the logical address 25
  - **Execution-time binding** occurs when reference is made to location in memory
  - Logical address bound to physical addresses

Dynamic relocation using a relocation register



Hardware Support for Relocation and Limit Registers

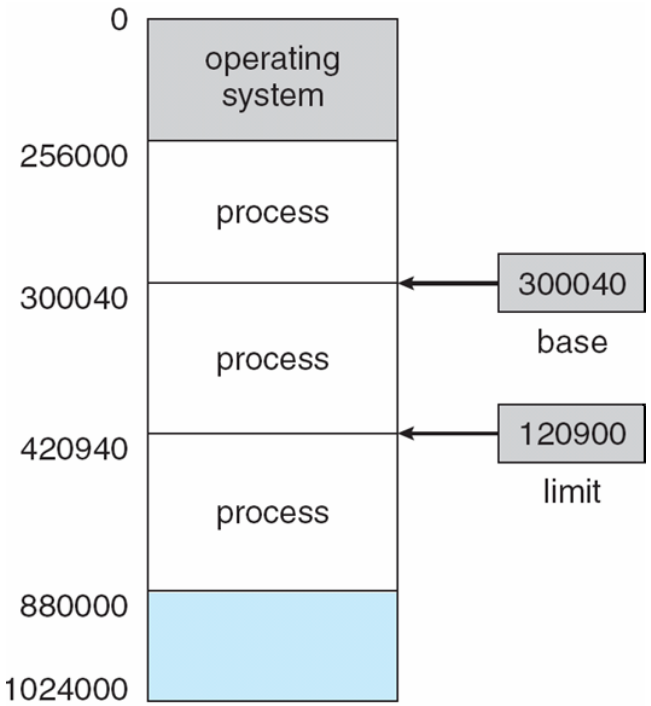
- **Relocation registers** used to protect user processes from each other, and from changing operating-system code and data
  - Relocation register contains value of smallest physical address
  - Limit register contains range of logical addresses – each logical address must be less than the limit register
  - Context switch
  - MMU maps logical address *dynamically*



- OS loads the reloc & limit reg.
- Privileged instruction

Contiguous Allocation

Multiple processes resides in memory



Contiguous Allocation

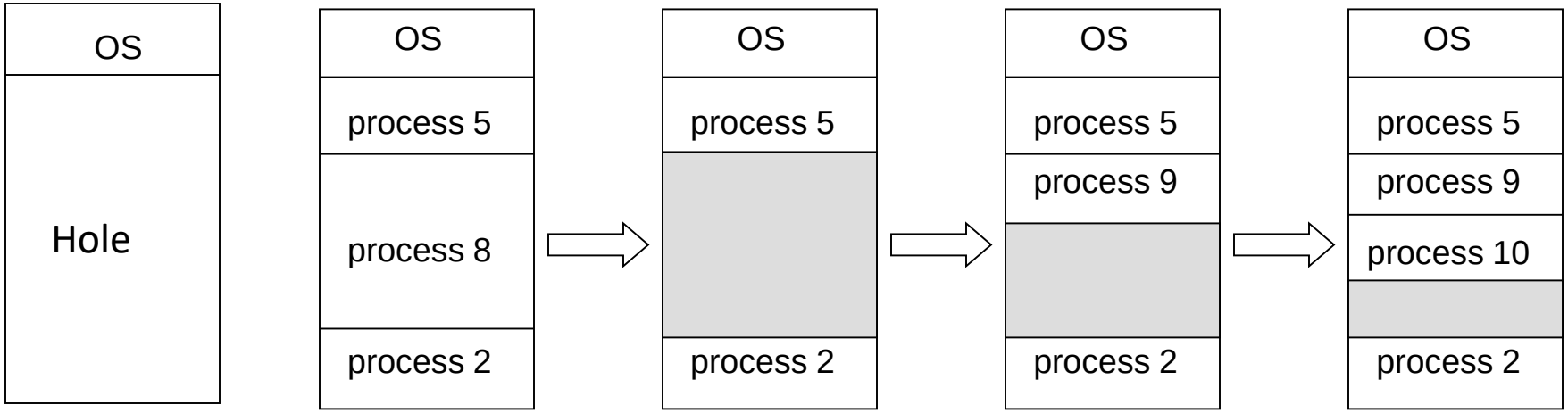
- Main memory usually divided into two partitions:
  - Resident operating system, usually held in high memory
  - User processes then held in low memory
  - Each process contained in **single contiguous section** of memory

Contiguous Allocation (Cont.)

- Multiple-partition allocation
  - Divide memory into several **Fixed size partition**
  - Each partition stores one process
  - Degree of multiprogramming limited by number of partitions
  - If a partition is free, load process from job queue
  - MFT (IBM OS/360)

Contiguous Allocation (Cont.)

- Multiple-partition allocation
  - **Variable partition scheme**
  - Hole – block of available memory; holes of various size are scattered throughout memory
  - Keeps a table of free memory
  - When a process arrives, it is allocated memory from a hole large enough to accommodate it
  - Process exiting frees its partition, adjacent free partitions combined
  - Operating system maintains information about:
    a) allocated partitions b) free partitions (hole)



Dynamic Storage-Allocation Problem

How to satisfy a request of size *n* from a list of free holes?

Dynamic storage allocation problem

- **First-fit**: Allocate the *first* hole that is big enough
- **Best-fit**: Allocate the *smallest* hole that is big enough; must search entire list, unless ordered by size
  - Produces the smallest leftover hole
- **Worst-fit**: Allocate the *largest* hole; must also search entire list
  - Produces the largest leftover hole

Fragmentation

- Processes loaded and removed from memory
  - Memory is broken into little pieces

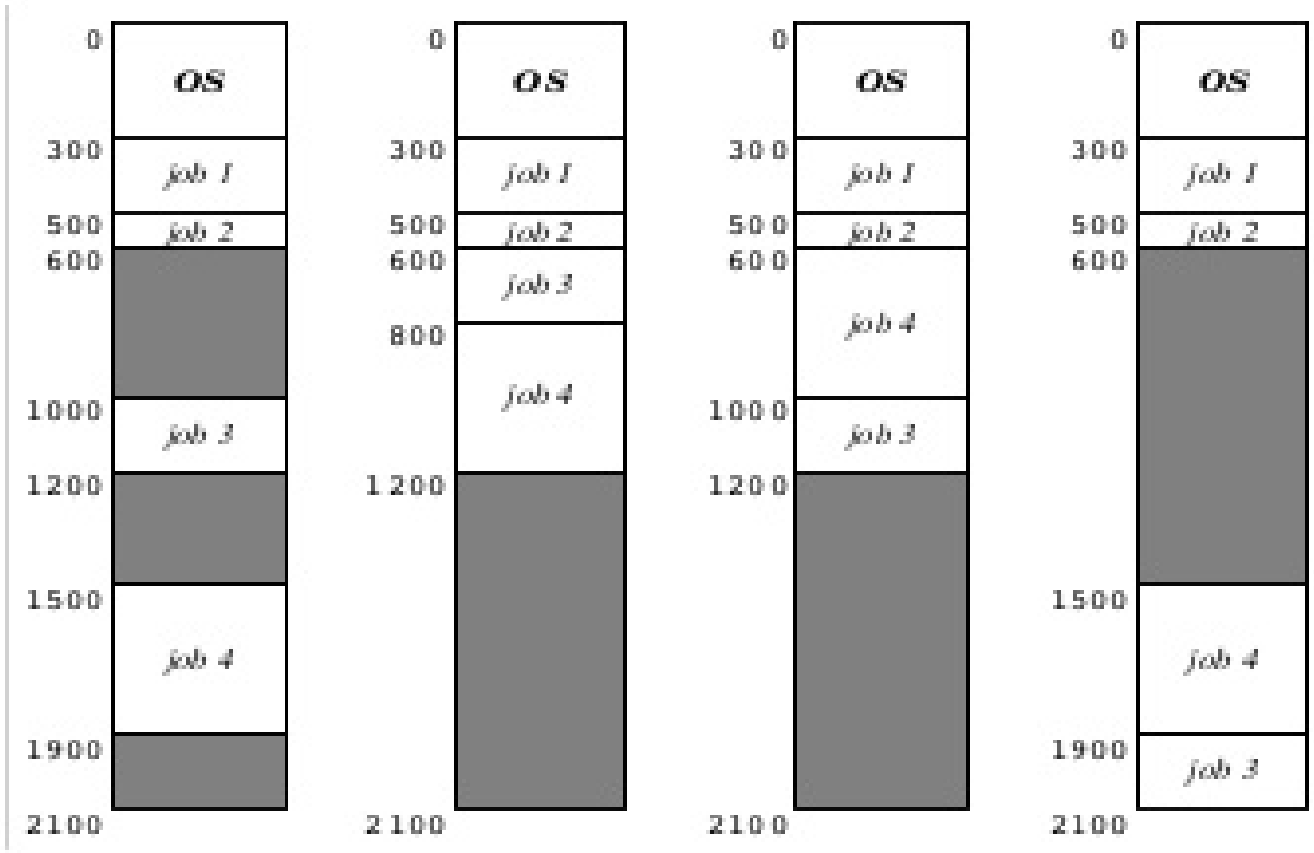
- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous

- First fit analysis reveals that given *N* blocks allocated, 0.5 *N* blocks lost to fragmentation
  - 1/3 may be unusable -> **50-percent rule**

Fragmentation (Cont.)

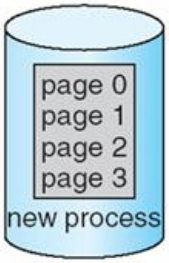
- Reduce external fragmentation by **compaction**
  - Shuffle memory contents to place all free memory together in one large block
  - Compaction is possible *only* if relocation is dynamic, and is done at execution time
    - Change relocation reg.
  - Cost

- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used


0
OS
300
job 1
500
job 2
600
1000
job 3
1200
1500
job 4
1900
2100
0
OS
300
job 1
500
job 2
600
job 3
800
job 4
1200
2100
0
OS
300
job 1
500
job 2
600
job 4
1000
job 3
1200
2100
0
OS
300
job 1
500
job 2
600
1500
job 4
1900
job 3
2100

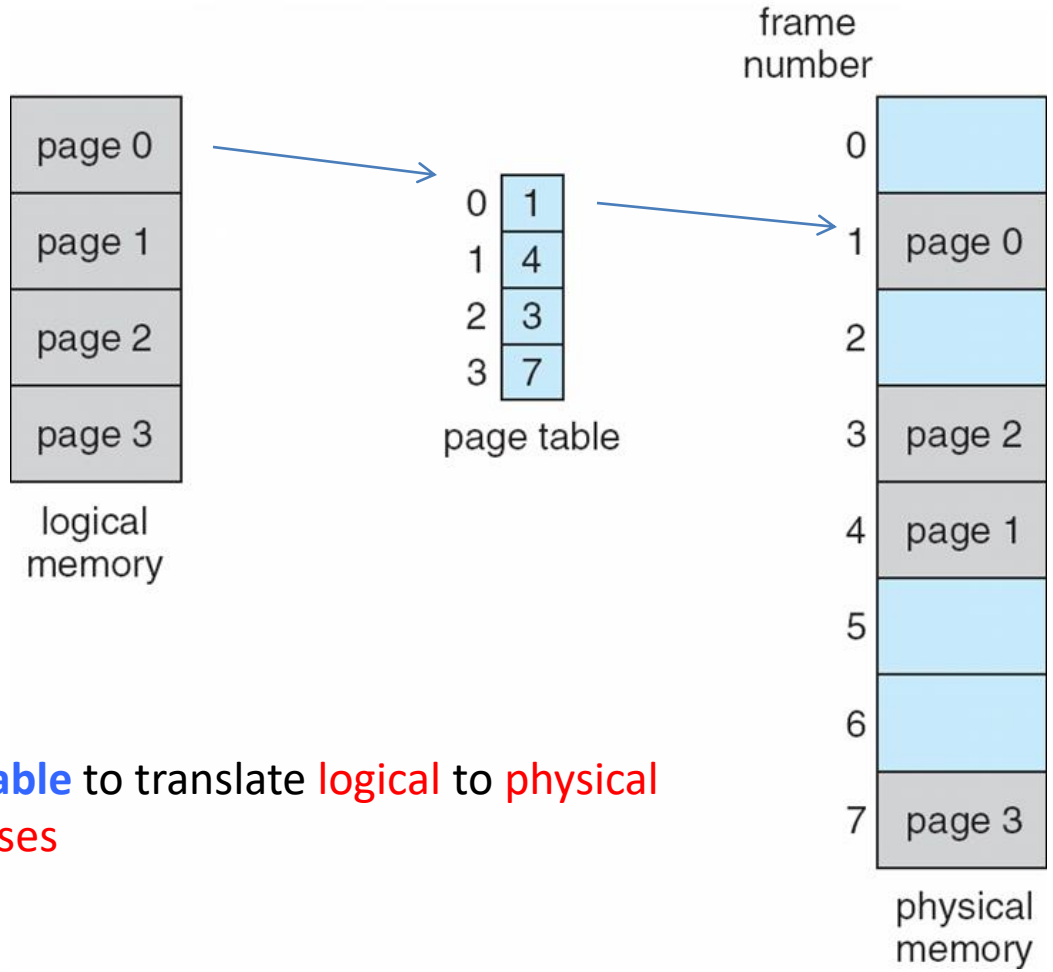
Paging

- Physical address space of a process can be **noncontiguous**;
  - process allocates physical memory whenever the latter is available
- Divide physical memory into fixed-sized blocks called **frames**
  - Size is power of 2, between 512 bytes and 16 Mbytes
- Divide logical memory into blocks of same size called **pages**
  - To run a program of size *N* pages, need to find *N* free frames and load program
- Backing store likewise split into pages



- Set up a **page table** to translate logical to physical addresses
- System keeps track of all free frames

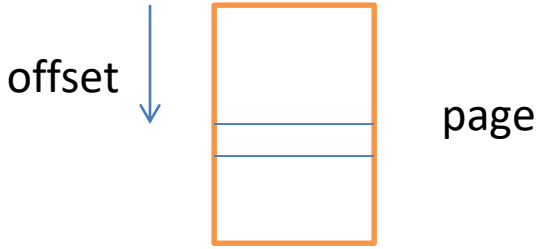
Paging Model of Logical and Physical Memory



page table to translate logical to physical addresses

Address Translation Scheme

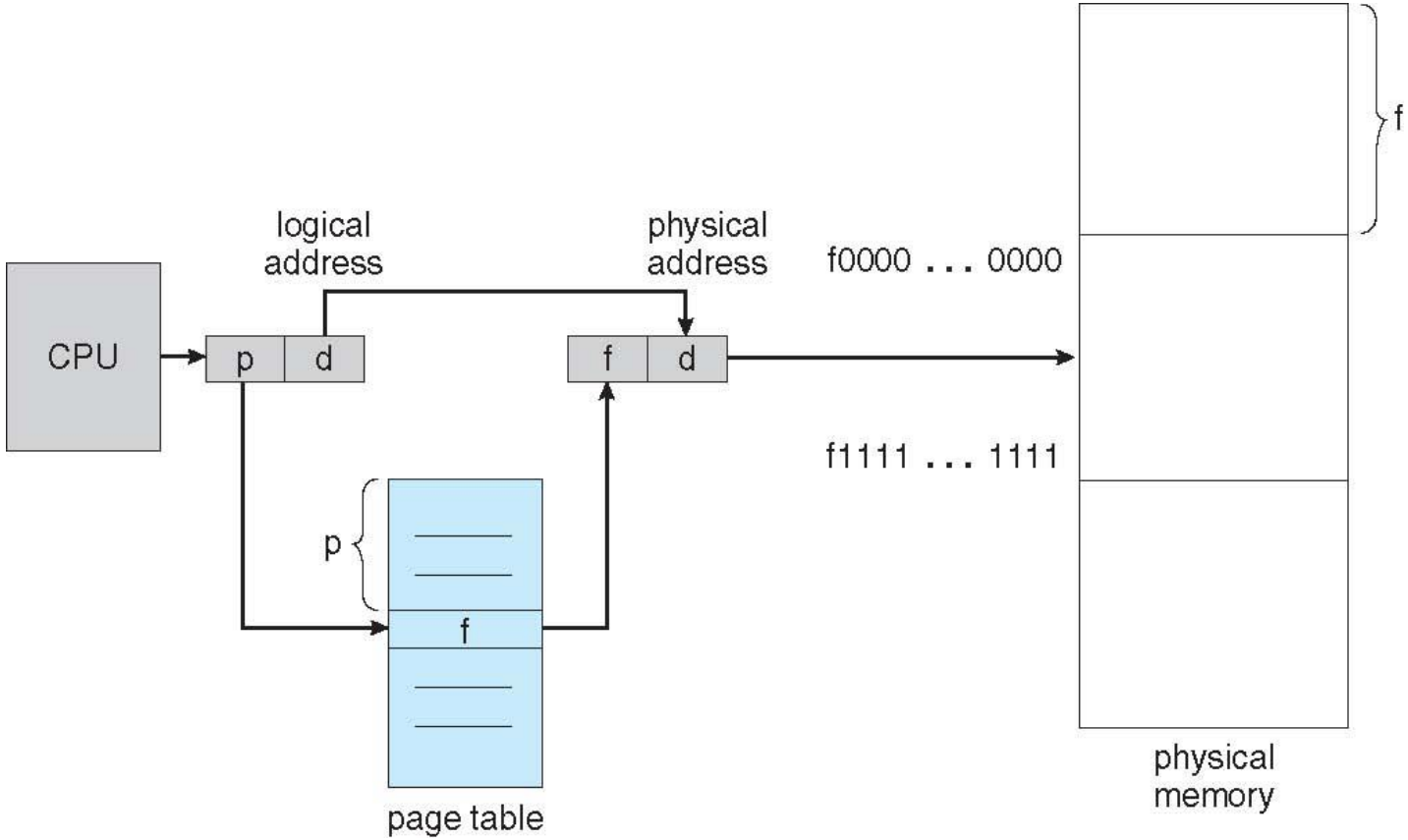
- Address generated by CPU is divided into:
  - **Page number (*p*)** – used as an index into a **page table**
    - which contains base address of each frame in physical memory
  - **Page offset (*d*)** – offset within a page
    - combined with base address to define the physical memory address that is sent to the memory unit



page number	page offset
p	*d*
m - n	*n*

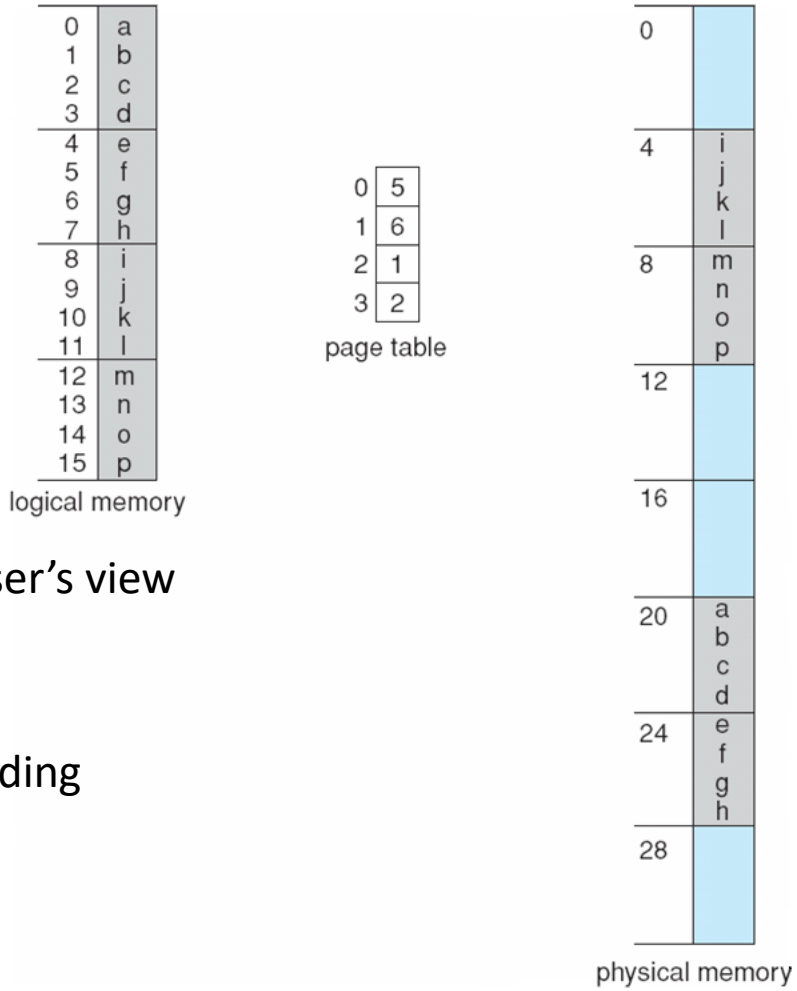
  - **For given logical address space 2^m and page size 2^n**

Paging Hardware



Paging Example

Logical address = 16
Page size=4
Physical memory=32



User's view

Run time address binding

n=2 and *m*=4 32-byte memory and 4-byte pages

Logical address 0
(0*4+0)
Physical address:
(5*4+0)=20

Logical address 3
(0*4+3)
Physical address:
(5*4+0)=23

Logical address 4
(1*4+0)
Physical address:
(6*4+0)=24

Logical address 13
(3*4+1)
Physical address:
(2*4+1)

Paging

- External fragmentation??
- Calculating internal fragmentation
  - Page size = 2,048 bytes
  - Process size = 72,766 bytes
  - 35 pages + 1,086 bytes
  - Internal fragmentation of 2,048 - 1,086 = 962 bytes
- So small frame sizes desirable?
  - But increases the page table size
  - Poor disk I/O
  - Page sizes growing over time
    - Solaris supports two page sizes – 8 KB and 4 MB
- User's view and physical memory now very different
  - user view=> process contains in single contiguous memory space
- By implementation process can only access its own memory
  - protection

Page table entries

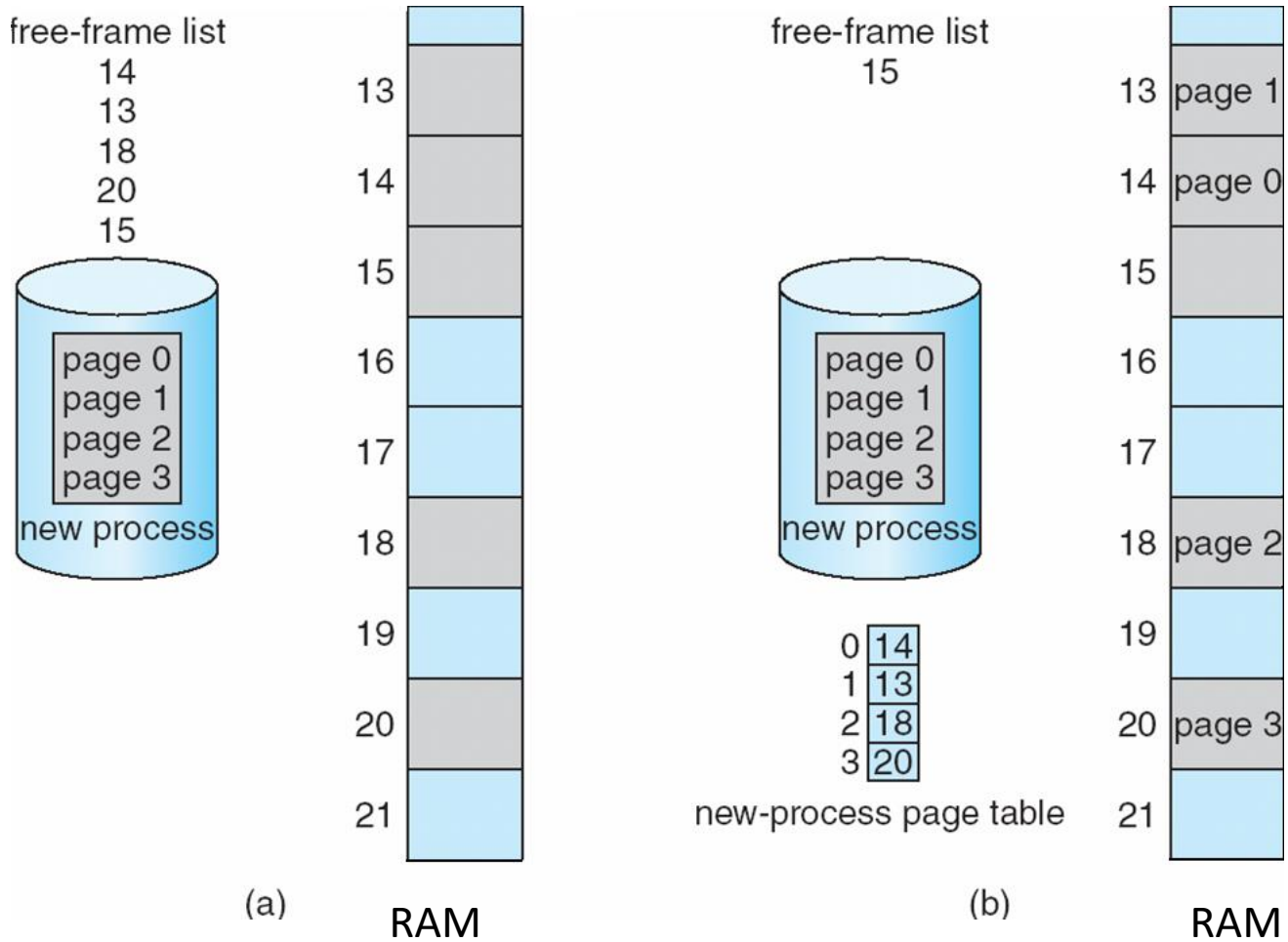
- Each page table entry 4 bytes (32 bits) long
- Each entry can point to 2^{32} page frames
- If each frame is 4 KB
- The system can address 2^{44} bytes (16TB) of physical memory

Virtual address space 16MB.
Page table size?

- Process P1 arrives
- Requires n pages => n frames must be available
- Allocate n frames to the process P1
- Create page table for P1

Frame table

Free Frames



Use's view

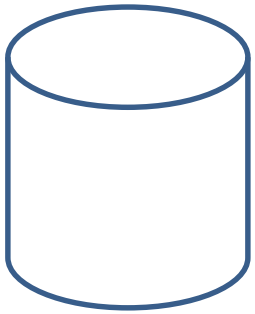
System's view

Before allocation

After allocation

Executable file and virtual address

a.out



Symbol table	
Name	address
SQR	0
SUM	4

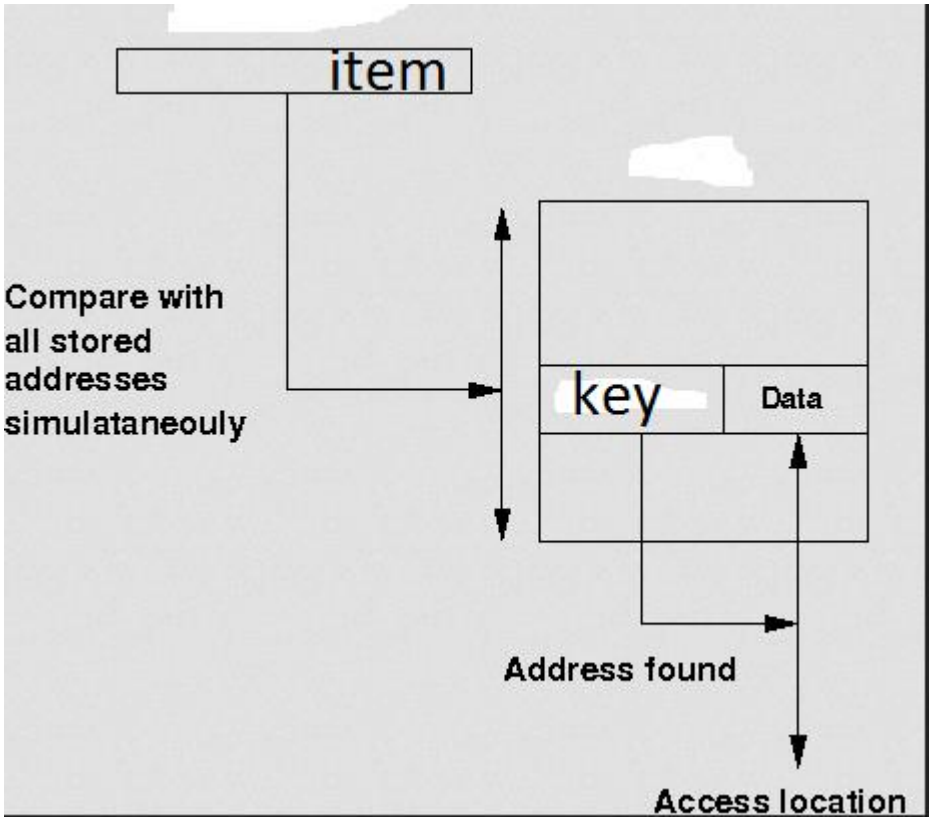
Virtual address space

```
TEXT    code
MAIN    movf    NUM_1,w     ; Get Number 1
        call    SQR         ; Square it
        movf    SQUARE+1,w  ; Get lower byte
        movwf   SUM+1       ; Is the low byte of sum
        movf    SQUARE,w    ; Get upper byte
        movwf   SUM         ; Is the high byte of sum
```

Implementation of Page Table

- For each process, Page table is kept in main memory
- **Page-table base register (PTBR)** points to the page table
- **Page-table length register (PTLR)** indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
  - One for the page table and one for the data / instruction
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**

Associative memory



Associative Memory

- Associative memory – parallel search

Page #	Frame #

- Address translation (p, d)
  - If p is in **associative register**, get frame # out
  - Otherwise get frame # from **page table in memory**

Implementation of Page Table

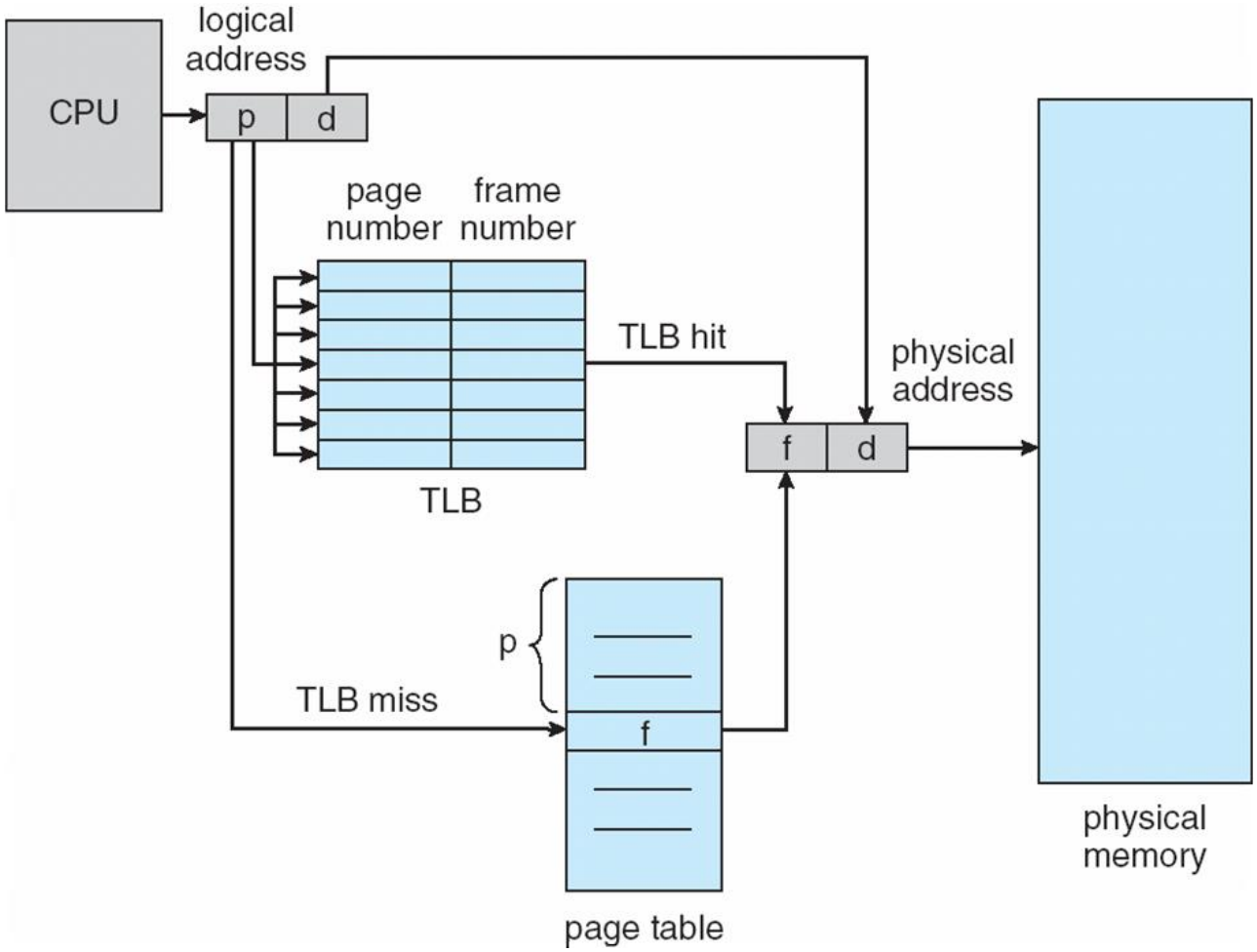
- For each process, Page table is kept in main memory
- **Page-table base register (PTBR)** points to the page table
- **Page-table length register (PTLR)** indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
  - One for the page table and one for the data / instruction
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**

- **TLBs typically small (64 to 1,024 entries)**

- On a **TLB miss**, value is loaded into the TLB for faster access next time
  - Replacement policies must be considered (LRU)
  - Some entries can be **wired down** for permanent fast access

- Some TLBs store **address-space identifiers (ASIDs)** in each TLB entry – uniquely identifies each process (PID) to provide address-space protection for that process
  - Otherwise need to flush at every context switch

Paging Hardware With TLB



Effective Access Time

- Associative Lookup = ε time unit
  - Can be < 10% of memory access time
- Hit ratio = α
  - Hit ratio – percentage of times that a page number is found in the associative registers; ratio related to size of TLB
- Consider α = 80%, ε = 20ns for TLB search, 100ns for memory access
- **Effective Access Time (EAT)**

$$EAT = (100 + \varepsilon)\,\alpha + (200 + \varepsilon)(1 - \alpha)$$

- Consider α = 80%, ε = 20ns for TLB search, 100ns for memory access
  - EAT = 0.80 x 120 + 0.20 x 220 = 140ns
- Consider better hit ratio -> α = 98%, ε = 20ns for TLB search, 100ns for memory access
  - EAT = 0.98 x 120 + 0.02 x 220 = 122ns

Memory Protection

- Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed
  - Can also add more bits to indicate page execute-only, and so on
- **Valid-invalid** bit attached to each entry in the page table:
  - "valid" indicates that the associated page is in the process' logical address space, and is thus a legal page
  - "invalid" indicates that the page is not in the process' logical address space
  - Or use PTLR
- Any violations result in a trap to the kernel

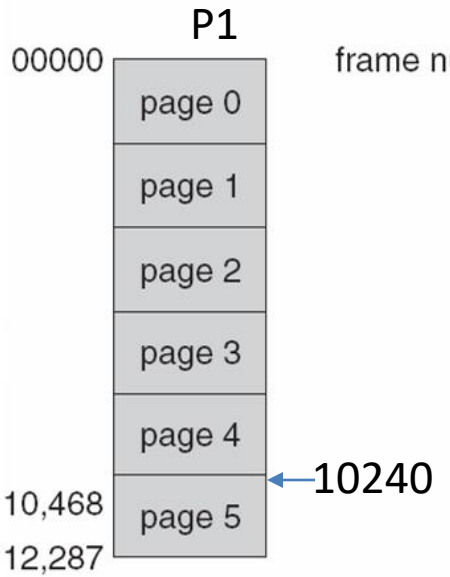
Valid (v) or Invalid (i)
Bit In A Page Table

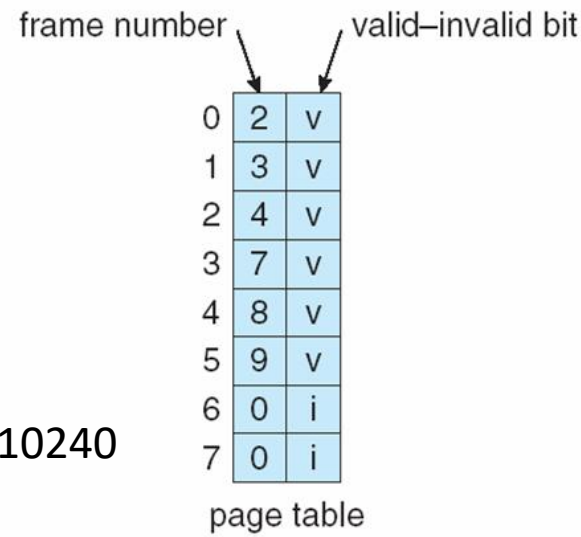
14 bit address space (0 to 16383)
Page size 2KB

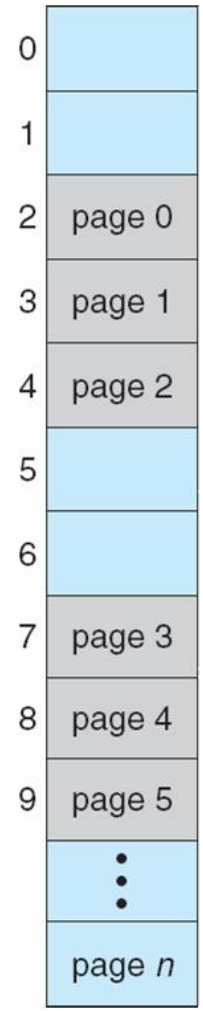
Process P1 uses only 0 to 10468

P2

Page 0
Page 1
Page 2
Page 3







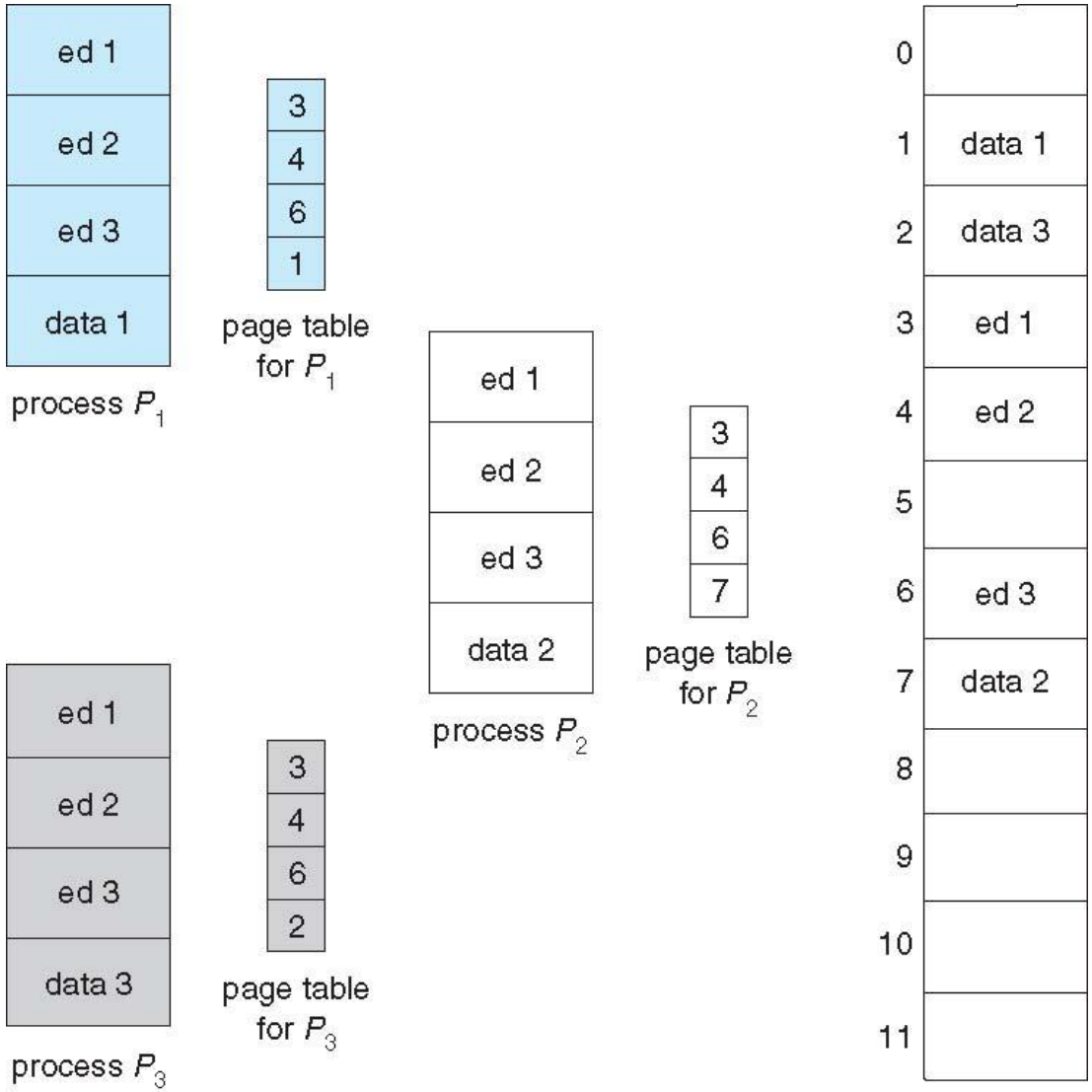
Internal fragmentation

Use of PTLR (length)

Shared Pages Example

- System with 40 users
  - Use common text editor
- Text editor contains 150KB code 50KB data (page size 50KB)
  - 8000KB!
- **Shared code**
  - One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, browsers)
    - Code never changes during execution
- Only one copy of the editor in the memory
- Total memory consumption
  - 40*50+150=2150KB

Shared Pages Example



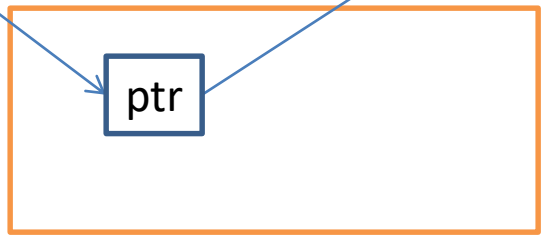
Data share: example

writer.c

```
int main()
{

      int shmid,f,key=3,i,pid;
      char *ptr;

      shmid=shmget((key_t)key,100,IPC_CREAT|0666);
      ptr=shmat(shmid,NULL,0);
      printf("shmid=%d ptr=%u\n",shmid, ptr);
      strcpy(ptr,"hello");
     i=shmdt((char*)ptr);

}
```

reader .c

```
int main()
{

      int shmid,f,key=3,i,pid;
      char *ptr;

      shmid=shmget((key_t)key,100,IPC_CREAT|0666);
      ptr=shmat(shmid,NULL,0);
      printf("shmid=%d ptr=%u\n",shmid, ptr);
      printf("\nstr %s\n",ptr);
}
```



Shared memory

Structure of the Page Table

- Memory requirement for page table can get huge using straight-forward methods
  - Consider a 32-bit logical address space as on modern computers
  - Page size of 4 KB (2^{12})
  - Page table would have 1 million entries 2^{20} (2^{32} / 2^{12})
  - If each entry is 4 bytes -> 4 MB of physical address space / memory for page table alone
    - That amount of memory used to cost a lot
    - Don't want to allocate that contiguously in main memory

- Hierarchical Paging

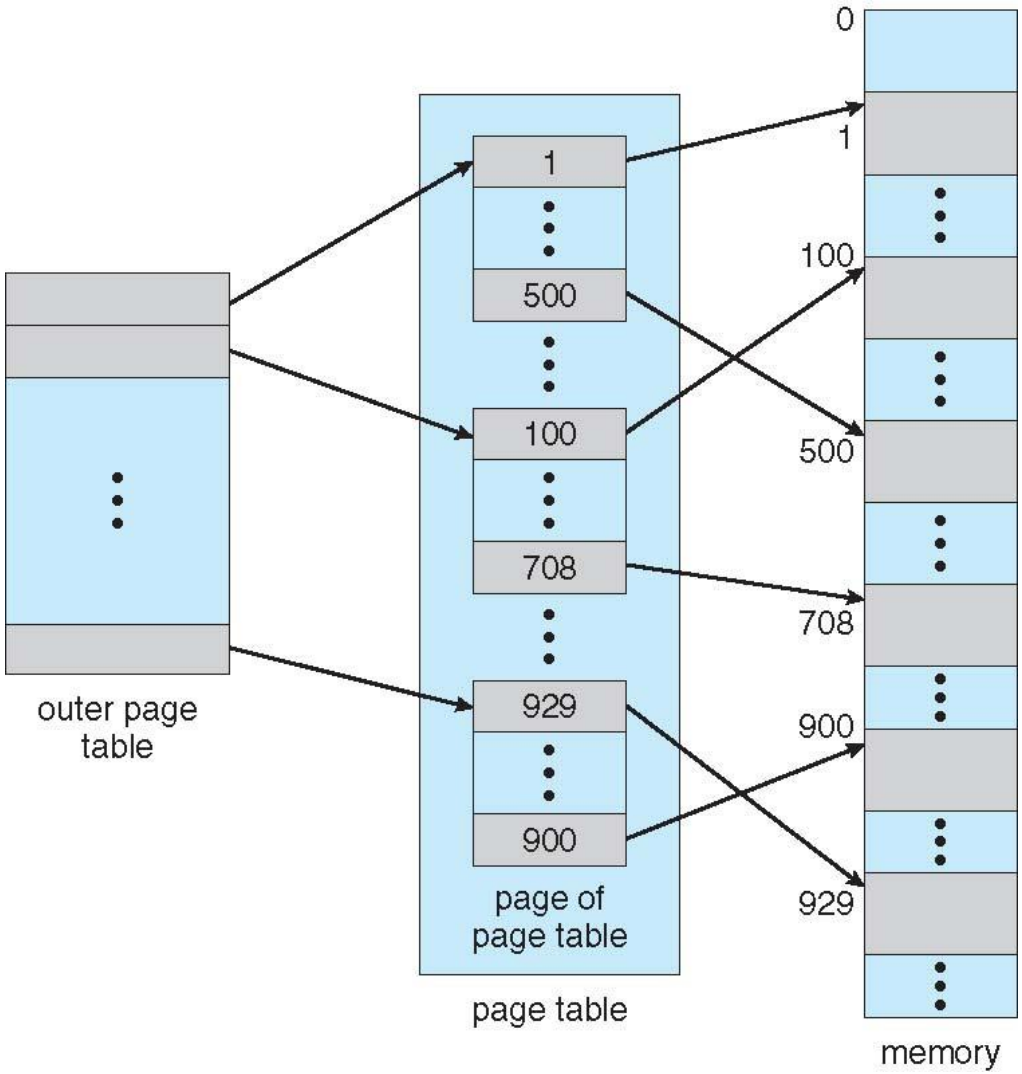
- Hashed Page Tables

- Inverted Page Tables

Hierarchical Page Tables

- Break up the page table into multiple pages
- We then page the page table, and store it in frame
- Create outer page table
- A simple technique is a two-level page table

Two-Level Page-Table Scheme



Two-Level Paging Example

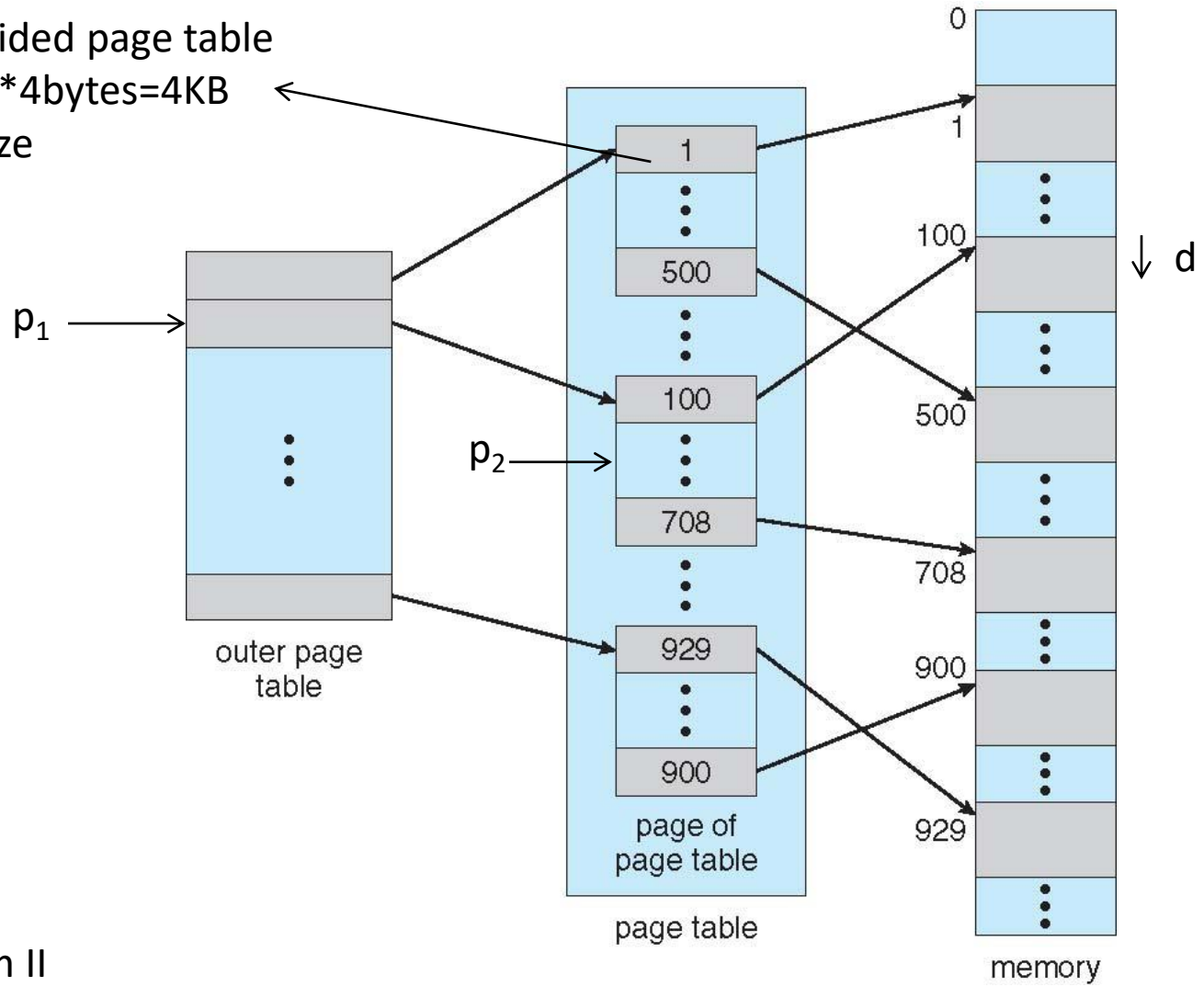
- A logical address (on 32-bit machine with 4KB page size) is divided into:
  - a page number consisting of 20 bits
  - a page offset consisting of 12 bits
- Since the page table is paged, the page number is further divided into:
  - a 10-bit page number
  - a 10-bit page offset
- Thus, a logical address is as follows:

page number		page offset
p_1	p_2	d
10	10	12

- where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the inner page table

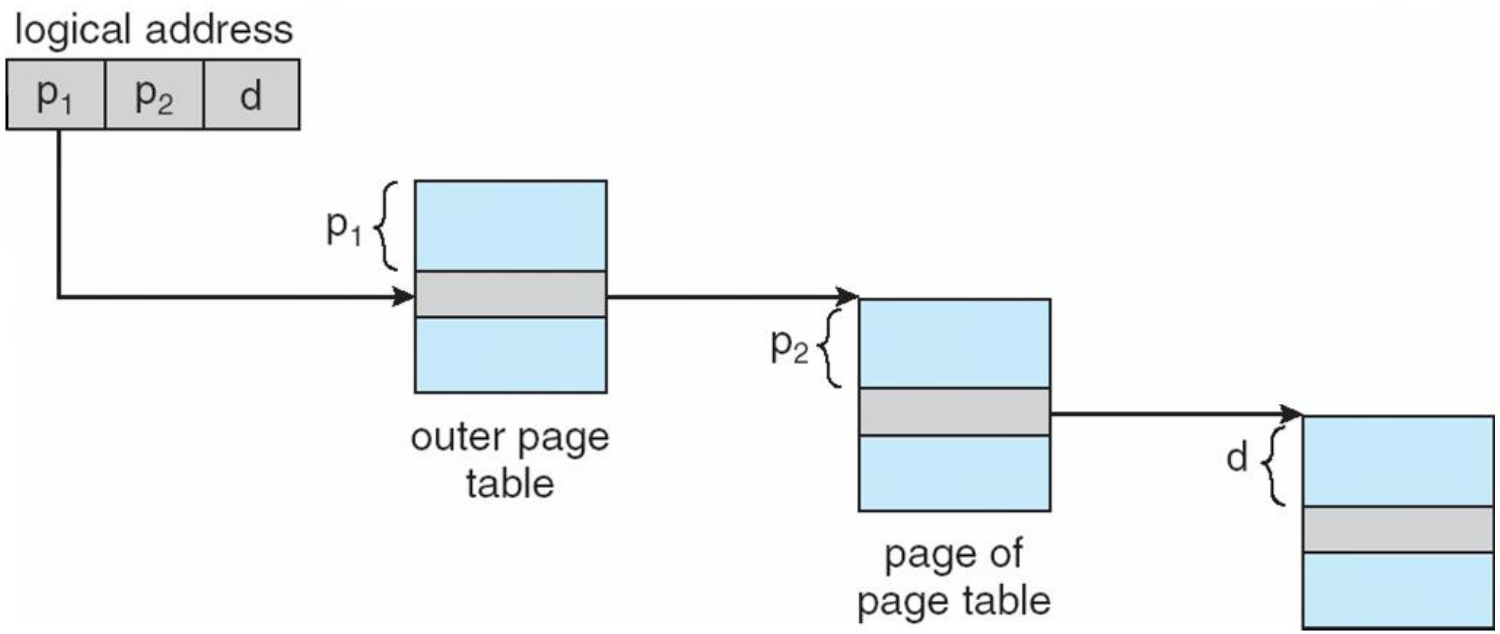
Two-Level Page-Table Scheme

Each divided page table size=2^{10} *4bytes=4KB =Page size



Pentium II

Address-Translation Scheme



Pentium II

64-bit Logical Address Space

- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
  - Then page table has 2^{52} entries
  - If two level scheme, inner page tables could be 2^{10} 4-byte entries
  - Address would look like

outer page	inner page	page offset
p_1	p_2	d
42	10	12

  - Outer page table has 2^{42} entries or 2^{44} bytes
  - One solution is to add a 2nd outer page table
  - But in the following example the 2nd outer page table is still 2^{34} bytes in size
    - And possibly 4 memory access to get to one physical memory location

Three-level Paging Scheme

outer page	inner page	offset
p_1	p_2	d
42	10	12

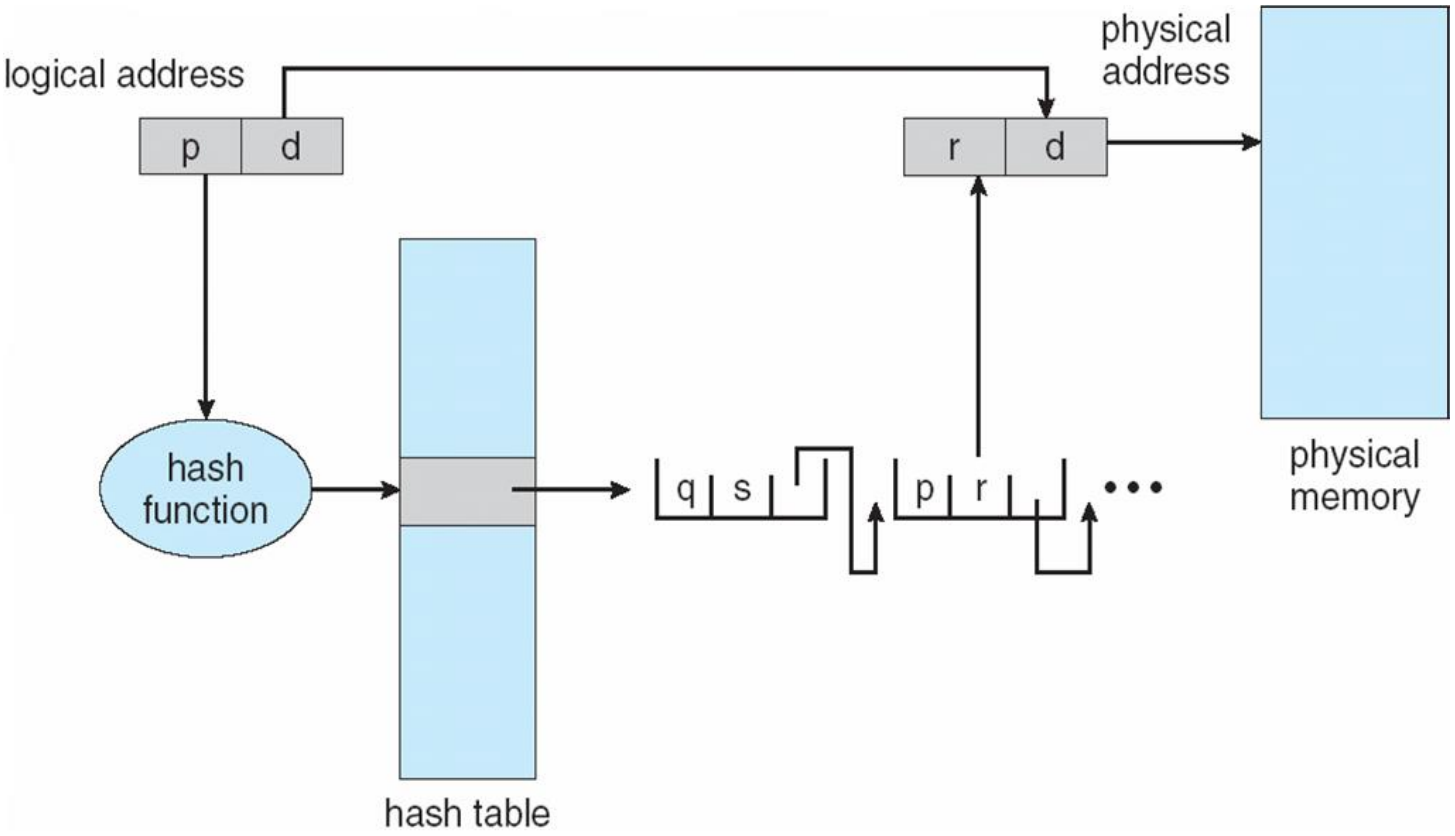
2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12

SPARC (32 bits), Motorola 68030 support three and four level paging respectively

Hashed Page Tables

- Common in virtual address spaces > 32 bits
- The page number is hashed into a page table
  - This page table contains a chain of elements hashing to the same location
- Each element contains (1) the page number (2) the value of the mapped page frame (3) a pointer to the next element
- Virtual page numbers are compared in this chain searching for a match
  - If a match is found, the corresponding physical frame is extracted

Hashed Page Table

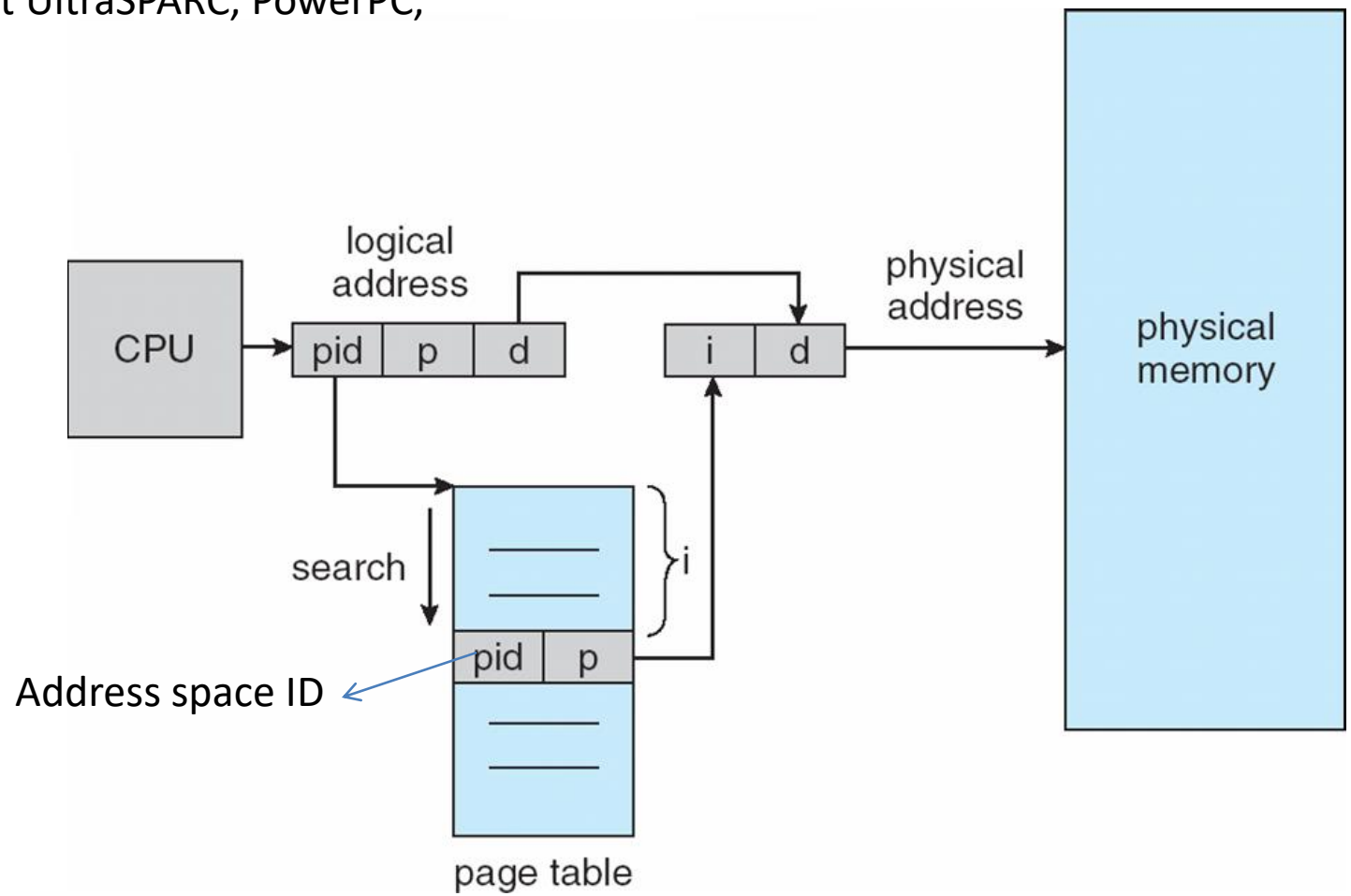


Inverted Page Table

- Rather than each process having a page table and keeping track of all possible logical pages, track all frames
- One entry for each frame
- Entry consists the **page number** stored in that frame, with **information about the process** that owns that page
- Decreases memory needed to store each page table,
  - but increases time needed to search the table when a page reference occurs

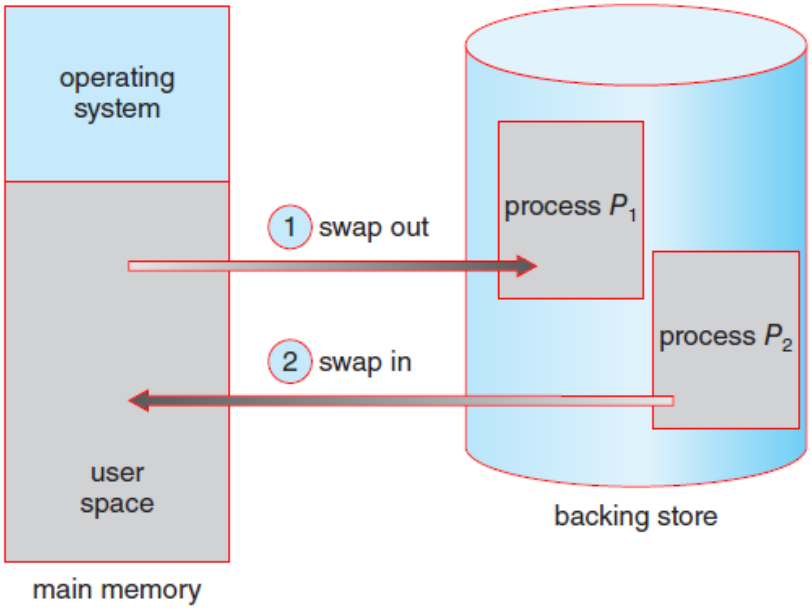
Inverted Page Table Architecture

64 bit UltraSPARC, PowerPC,



Swapping

A process, or a portion of a process, can be swapped temporarily out of memory to a backing store and then brought back into memory for continued execution



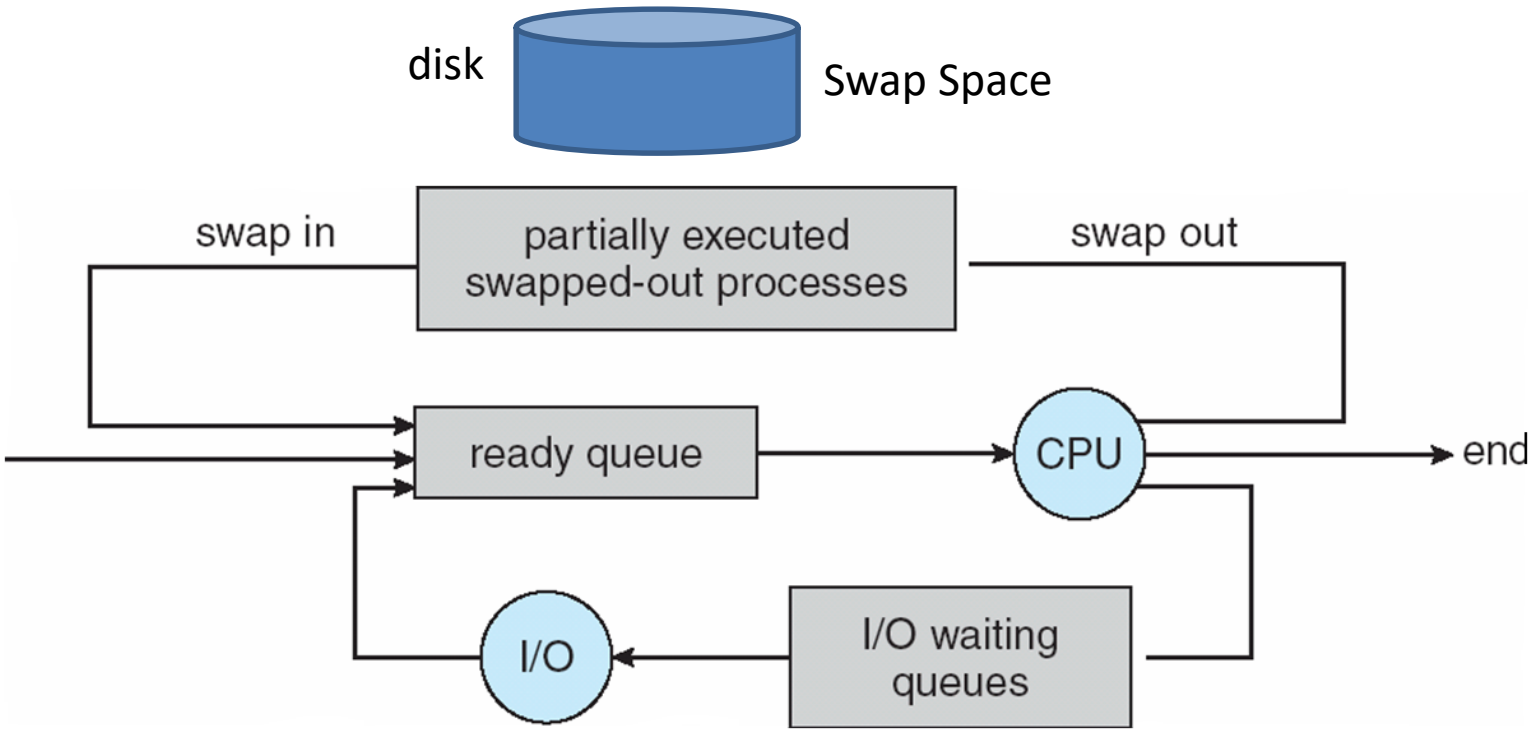
Expensive

Advantages :

Allows physical memory to be oversubscribed, so that the system can accommodate more processes than there is actual physical memory to store them.

Idle or mostly idle processes are good candidates for swapping; any memory that has been allocated to these inactive processes can then be dedicated to active processes.

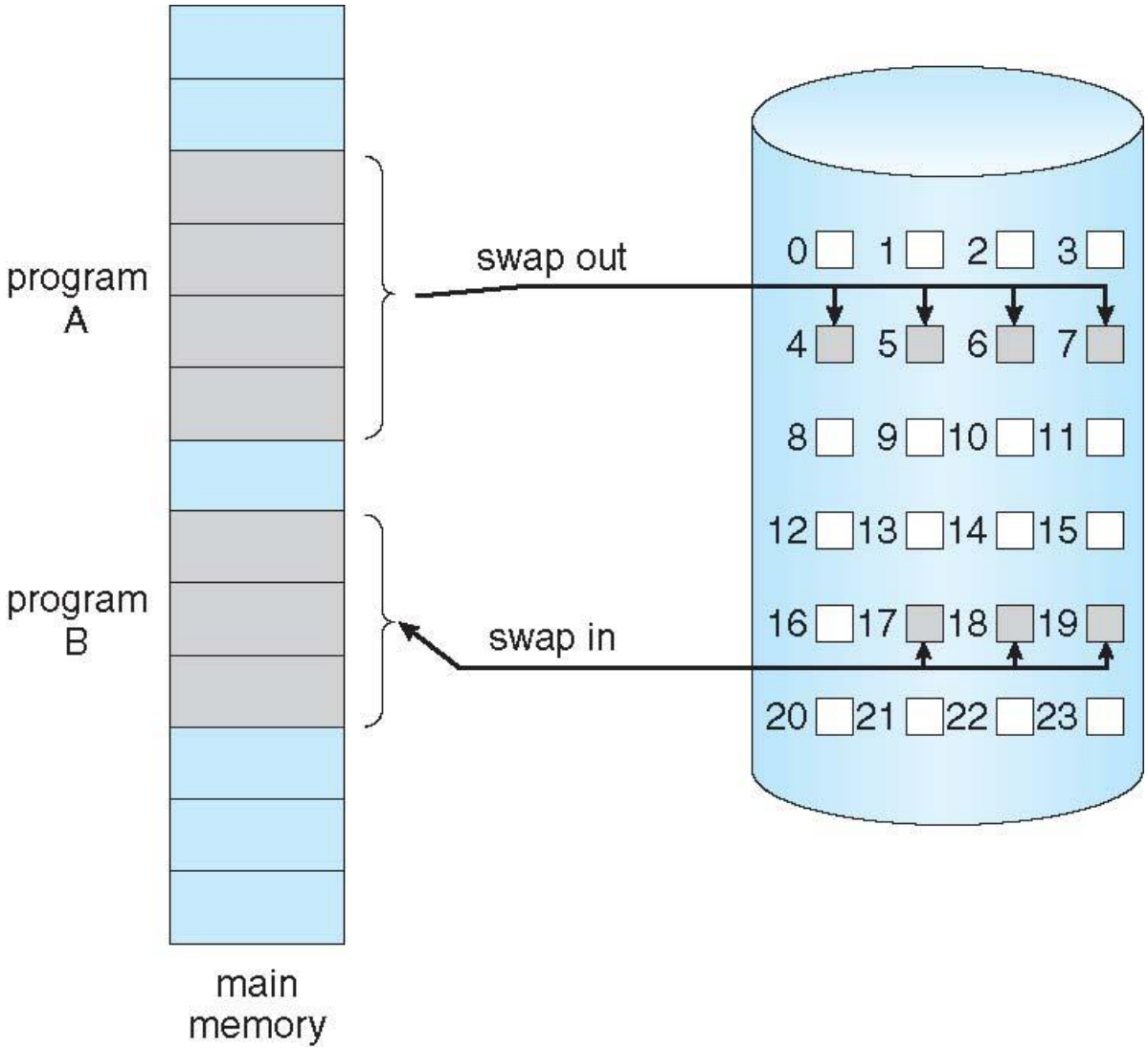
Medium Term Scheduling



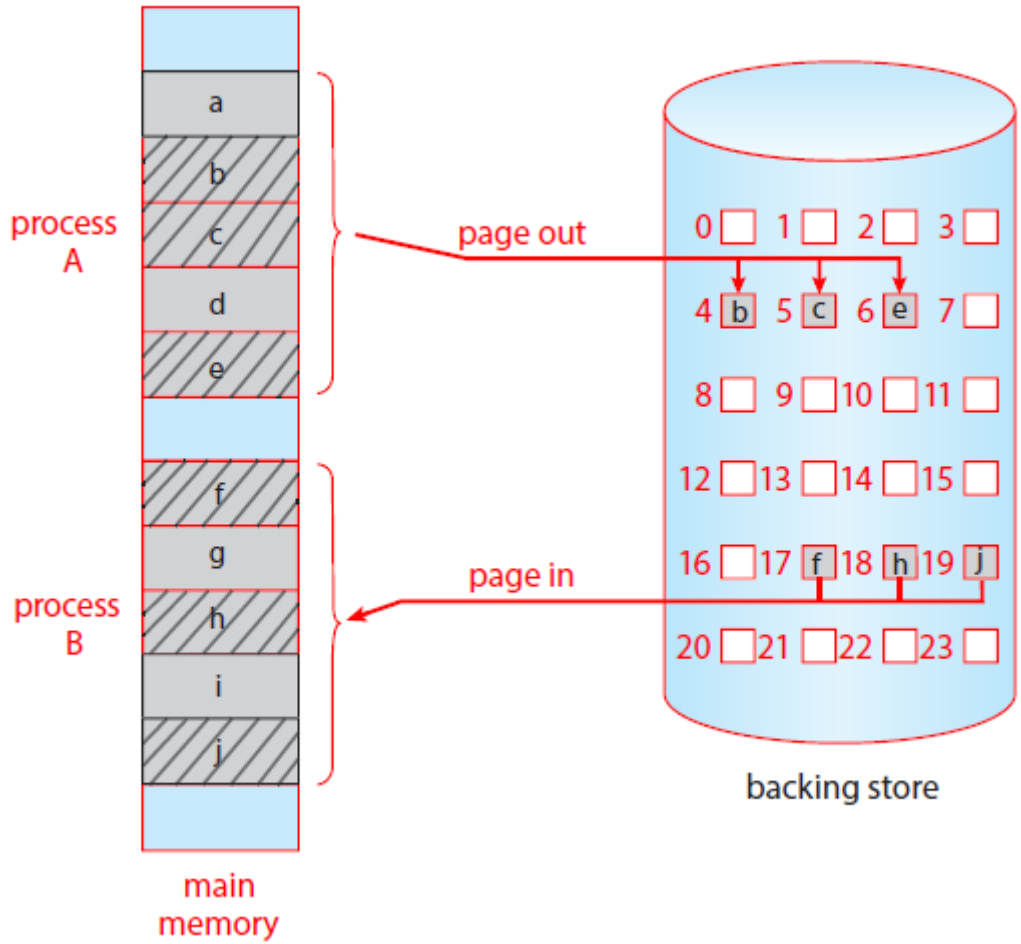
Swapper

Swapping with Paging

- When we want to execute a process, swap in
- Instead of swap in entire process, load page
- Pager



Swapping with Paging



Segmentation

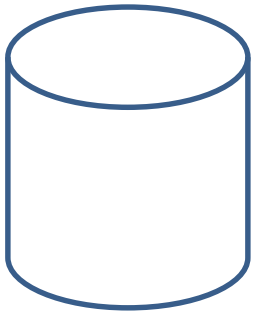
- Memory-management scheme that supports user view of memory
- A program is a collection of segments
  - A segment is a logical unit such as:

    main program
    procedure
    function
    method
    object
    local variables, global variables
    common block
    stack
    symbol table
    arrays

Compiler generates the segments
Loader assign the seg#

Executable file and virtual address

a.out

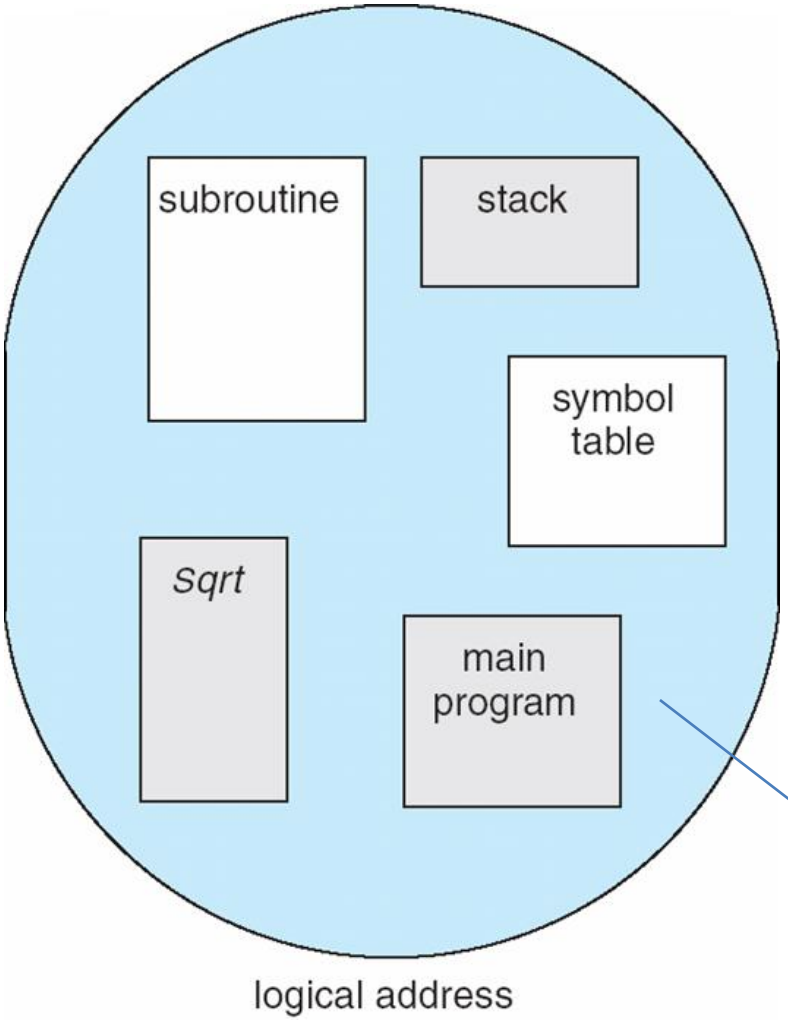


Symbol table	
Name	address
SQR	0
SUM	4

Virtual address space

```
TEXT    code
MAIN    movf    NUM_1,w     ; Get Number 1
        call    SQR         ; Square it
        movf    SQUARE+1,w  ; Get lower byte
        movwf   SUM+1       ; Is the low byte of sum
        movf    SQUARE,w    ; Get upper byte
        movwf   SUM         ; Is the high byte of sum
```

User's View of a Program



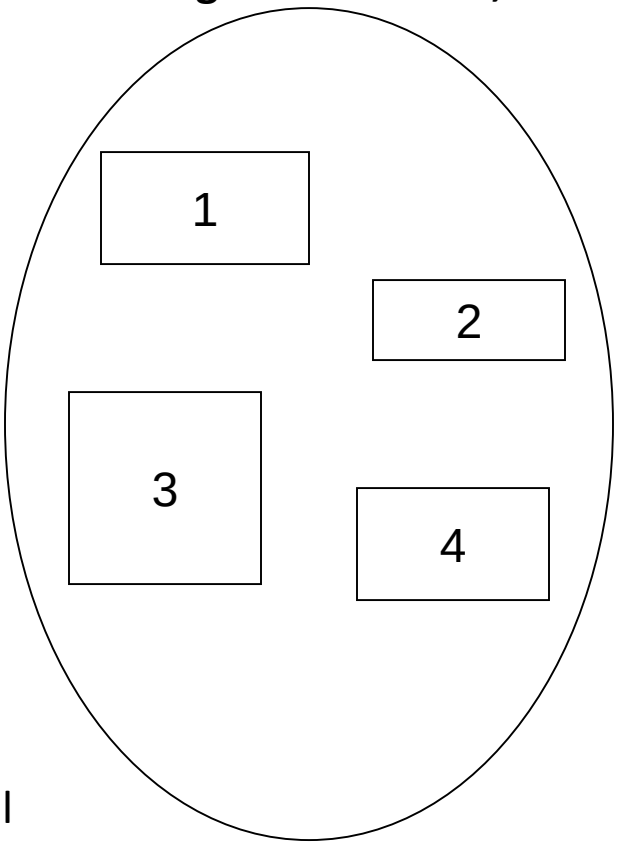
User specifies each address by two quantities
(a) Segment name
(b) Segment offset

Logical address contains the tuple
<segment#, offset>

- Variable size segments without order
- Length=> purpose of the program
- Elements are identified by offset

Logical View of Segmentation

Logical address **<segment-number, offset>**



Logical address space

user space

physical memory space

- Long term scheduler finds and allocates memory for all segments of a program
- Variable size partition scheme

Windows XP Memory Usage

Segment	First Address	Last Address	Size
Code	401000x	403000x	002000x ~ 8 Kbytes
Static (Global) Data	403000x	703000x	300000x ~ 3 megabytes
Heap	760000x	3A261000x	39800000x ~ 950 megabytes
Stack	22EF00x	16EF00x	1C0000x ~ 2 megabyte

LINUX Memory Usage

Segment	First Address	Last Address	Size
Code	8048400x	8049900x	001500x ~ 6 Kbytes
Static (Global) Data	8049A00x	8349A00	300000x ~ 3 megabytes
Heap	B7EE,B000x	01CE,4000x	B6000000x ~ 3 gigabytes
Stack	BFFB,7334x	29BA,91E0x	9640,0000x ~ 2.5 gigabyte

Memory image

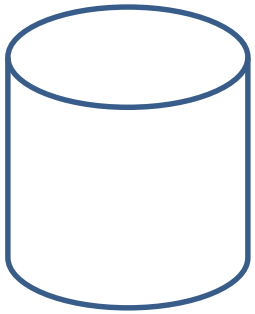
```
0x08048368 <main+0>:    55                  push   %ebp
0x08048369 <main+1>:    89 e5               mov    %esp,%ebp
0x0804836b <main+3>:    83 ec 08            sub    $0x8,%esp
0x0804836e <main+6>:    83 e4 f0            and    $0xfffffff0,%esp
0x08048371 <main+9>:    b8 00 00 00 00      mov    $0x0,%eax
0x08048376 <main+14>:   83 c0 0f            add    $0xf,%eax
0x08048379 <main+17>:   83 c0 0f            add    $0xf,%eax
0x0804837c <main+20>:   c1 e8 04            shr    $0x4,%eax
0x0804837f <main+23>:   c1 e0 04            shl    $0x4,%eax
0x08048382 <main+26>:   29 c4               sub    %eax,%esp
0x08048384 <main+28>:   83 ec 0c            sub    $0xc,%esp
0x08048387 <main+31>:   68 c0 84 04 08      push   $0x80484c0
0x0804838c <main+36>:   e8 1f ff ff ff      call   0x80482b0
0x08048391 <main+41>:   83 c4 10            add    $0x10,%esp
0x08048394 <main+44>:   e8 02 00 00 00      call   0x804839b <b>
```

```
void b();
void c();
int  main( )
{
   printf( "Hello from main\n");
   b();
}
// This routine reads the opcodes from memory and prints them out.
void  b()
 {
    char  *moving;

    for ( moving = (char *)(&main); moving < (char *)(&c); moving++ )
       printf( "Addr = 0x%x, Value = %2x\n", (int)(moving), 255 & (int)*moving );
 }
 void  c()
 {
 }
```

```
0x0804839b <b+0>:       55                  push   %ebp
0x0804839c <b+1>:       89 e5               mov    %esp,%ebp
0x0804839e <b+3>:       83 ec 08            sub    $0x8,%esp
0x080483a1 <b+6>:       c7 45 fc 68 83 04 08 movl  $0x8048368,0xfffffffc(%ebp)
0x080483a8 <b+13>:      81 7d fc d9 83 04 08 cmpl  $0x80483d9,0xfffffffc(%ebp)
0x080483af <b+20>:      73 26               jae    0x80483d7 <b+60>
0x080483b1 <b+22>:      83 ec 04            sub    $0x4,%esp
0x080483b4 <b+25>:      8b 45 fc            mov    0xfffffffc(%ebp),%eax
0x080483b7 <b+28>:      0f be 00            movsbl (%eax),%eax
0x080483ba <b+31>:      25 ff 00 00 00      and    $0xff,%eax
```

Executable file and virtual address

a.out



Symbol table	
Name	address
SQR	0
SUM	4

Virtual address space

Paging view

0	Load	0
4	ADD	4

Segmentation view

<CODE, 0>	Load	<ST,0>
<CODE, 2>	ADD	<ST,4>

Segmentation Architecture

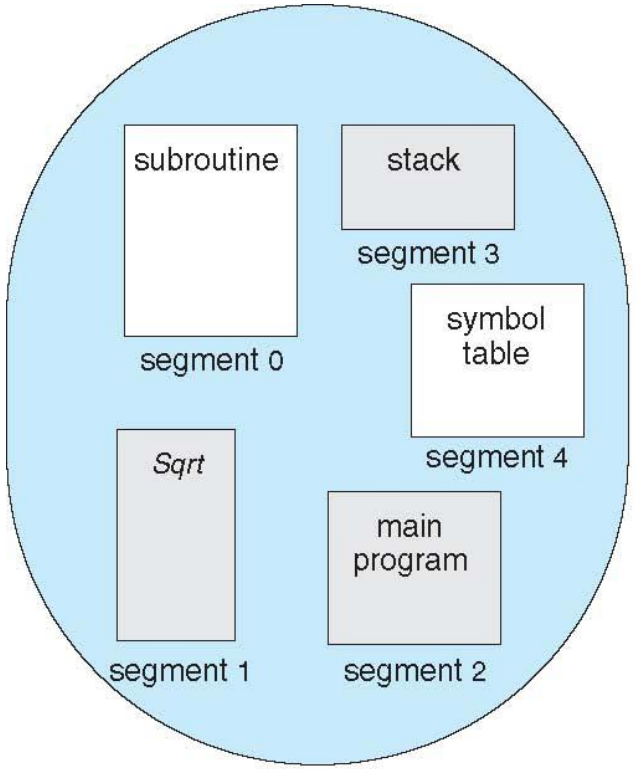
- Logical address consists of a two tuple:

  <segment-number, offset>

- **Segment table** – maps two-dimensional logical address to physical address;
- Each table entry has:
  - **base** – contains the starting physical address where the segments reside in memory
  - **limit** – specifies the length of the segment
- **Segment-table base register (STBR)** points to the segment table's location in memory
- **Segment-table length register (STLR)** indicates number of segments used by a program;

  segment number ***s*** is legal if ***s*** < **STLR**

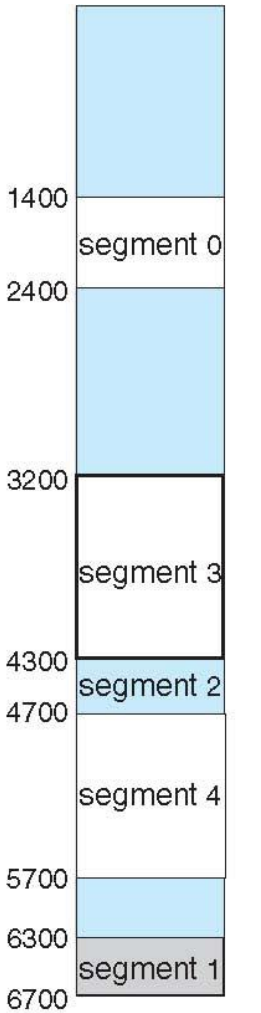
Example of Segmentation



logical address space

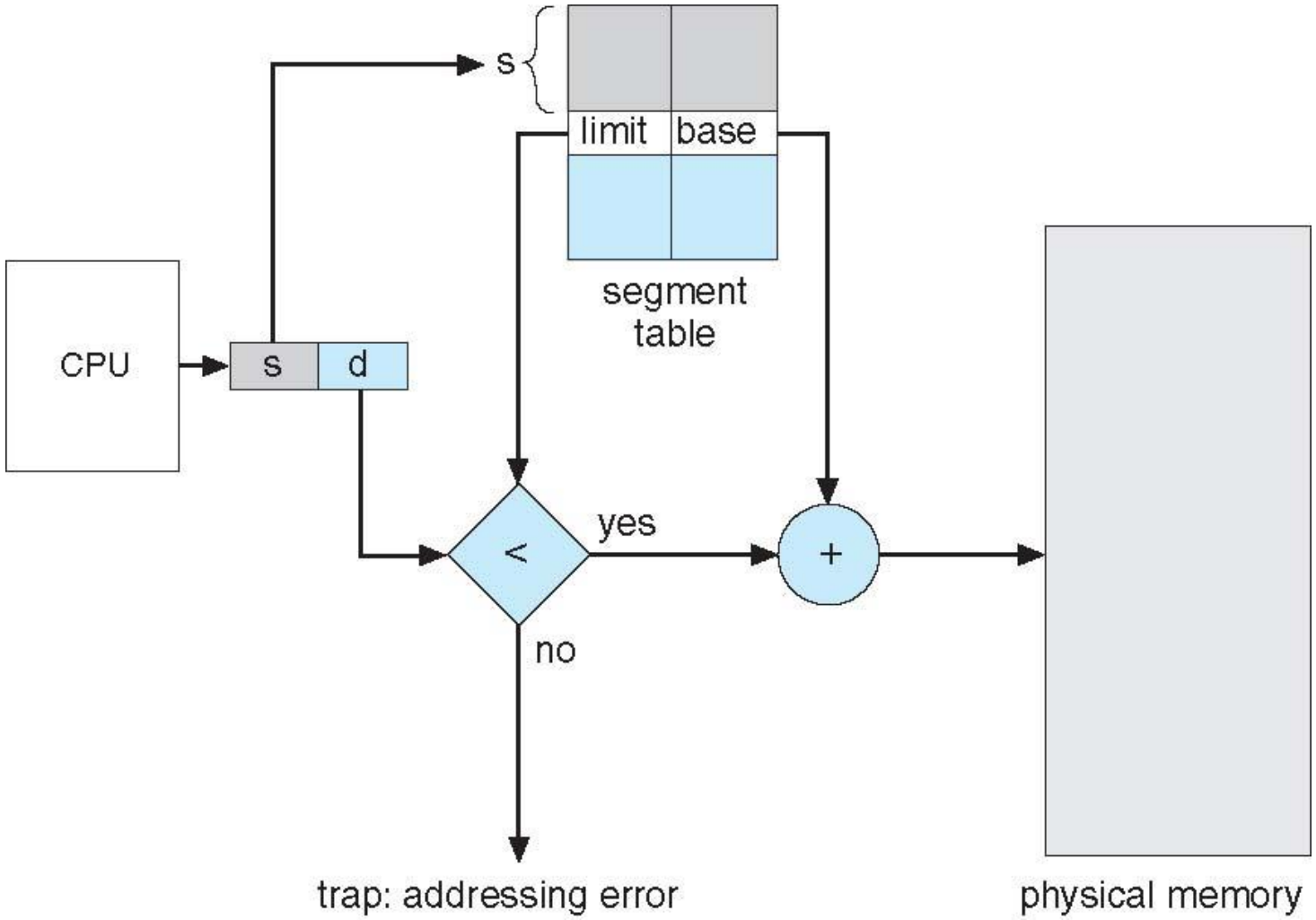
	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table

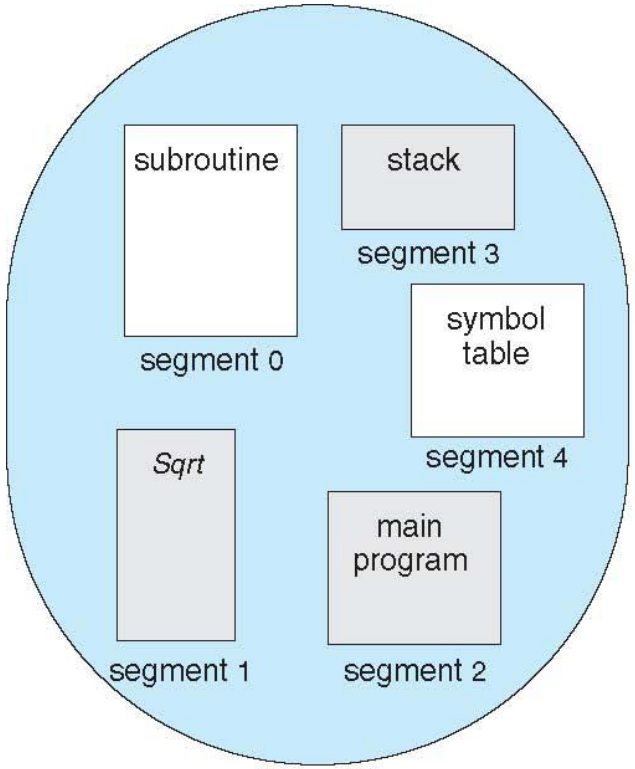


physical memory

Segmentation Hardware



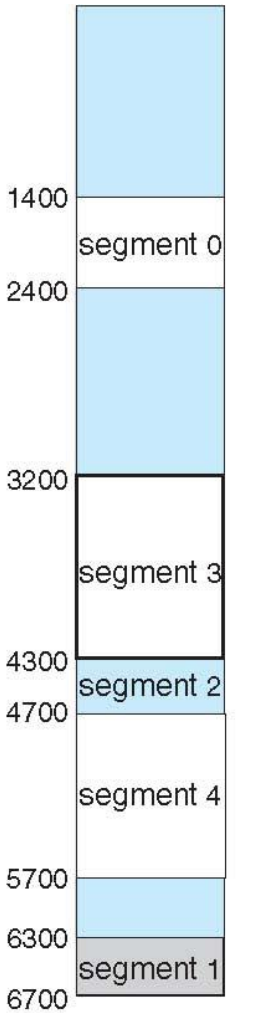
Example of Segmentation



logical address space

	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table



physical memory

Segmentation Architecture

- Protection
- Protection bits associated with segments
  - With each entry in segment table associate:
    - validation bit = 0 $\Rightarrow$ illegal segment
    - read/write/execute privileges
- Code sharing occurs at segment level
- Since segments vary in length, memory allocation is a dynamic storage-allocation problem
  - Long term scheduler
  - First fit, best fit etc
- Fragmentation

Segmentation with Paging

Key idea:

Segments are splitted into multiple pages

Each page is loaded into frames in the memory

Segmentation with Paging

- Supports both segmentation and segmentation with paging
  - Each segment can be 4 GB
  - Up to 16 K segments per process
  - <selector(16), offset (32)>
  - Divided into two partitions
    - First partition of up to 8 K segments are private to process (kept in **local descriptor table LDT**)
    - Second partition of up to 8K segments shared among all processes (kept in **global descriptor table GDT**)

S(13)	G(1)	P(2)

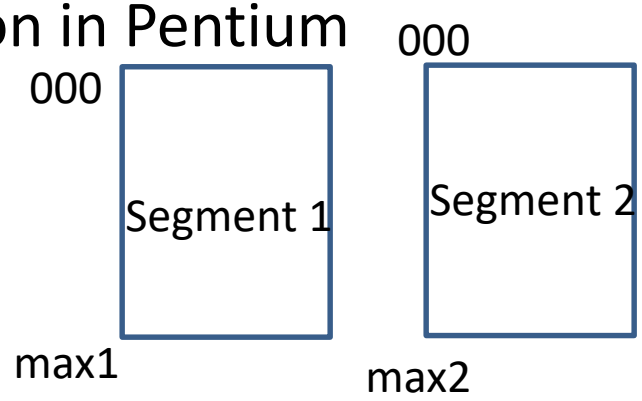
- CPU generates logical address (six Segment Reg.)
  - Given to segmentation unit
    - Which produces linear **(logical)** addresses
  - Physical address 32 bits
  - Linear address given to paging unit
    - Which generates physical address in main memory
    - Paging units form equivalent of MMU
    - Pages sizes can be 4 KB

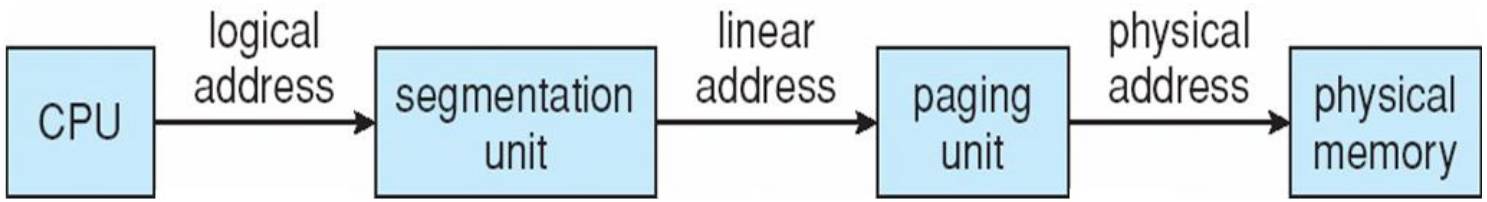
Intel 80386

IBM OS/2

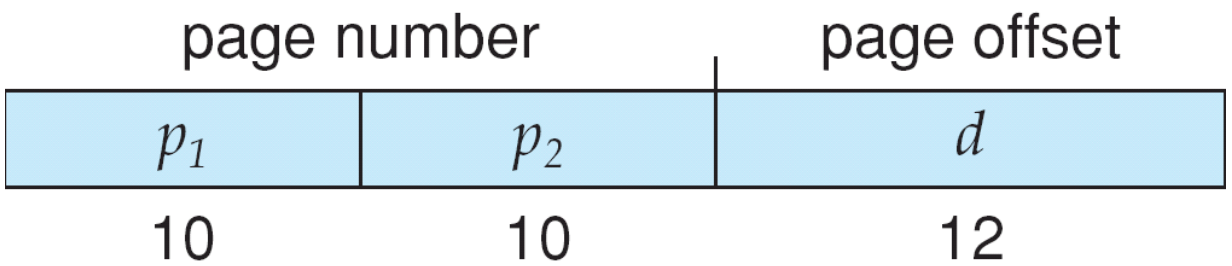
Logical to Physical Address Translation in Pentium

Each segment works as a separate logical address space

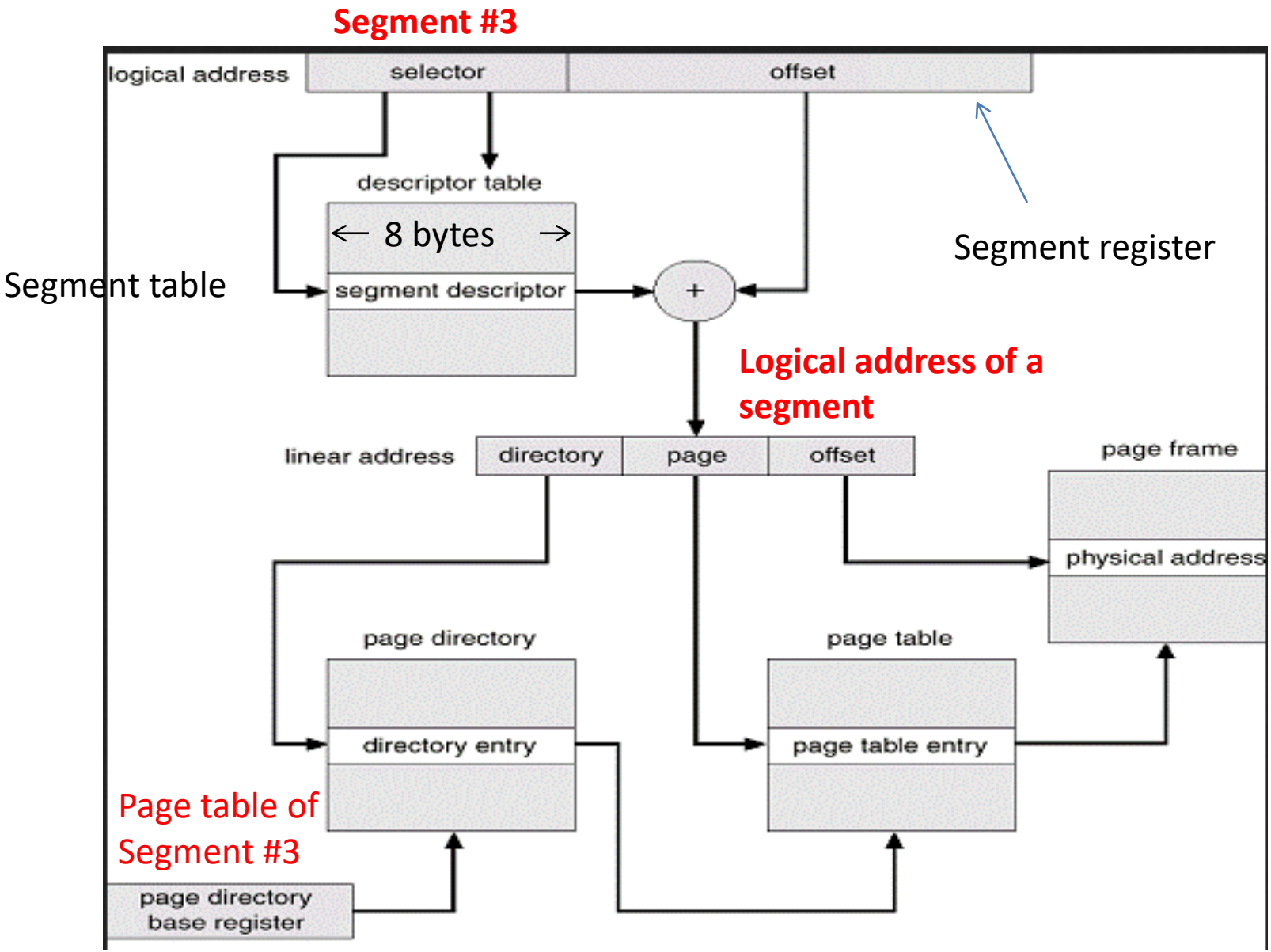




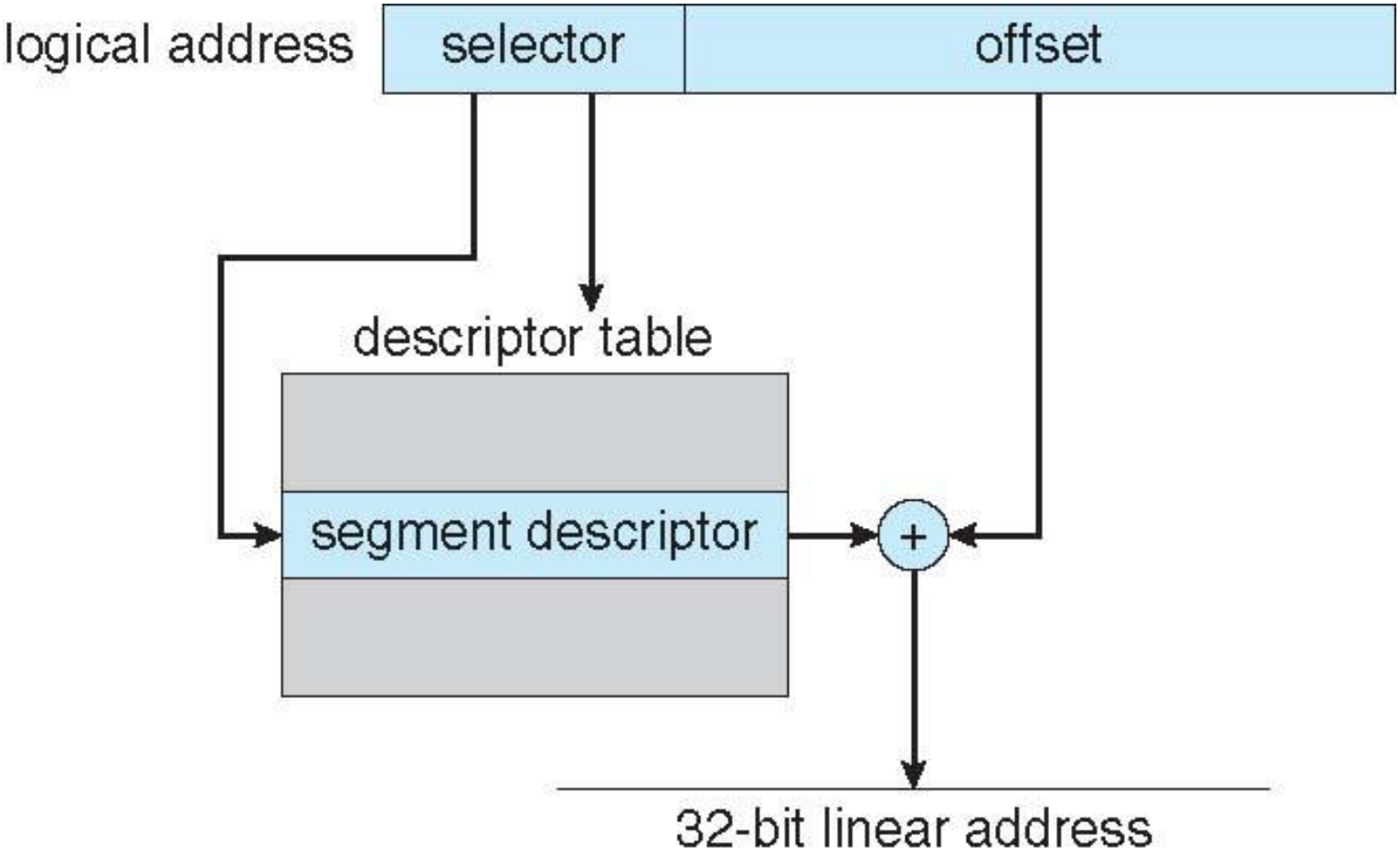
Page table=2^{20} entries



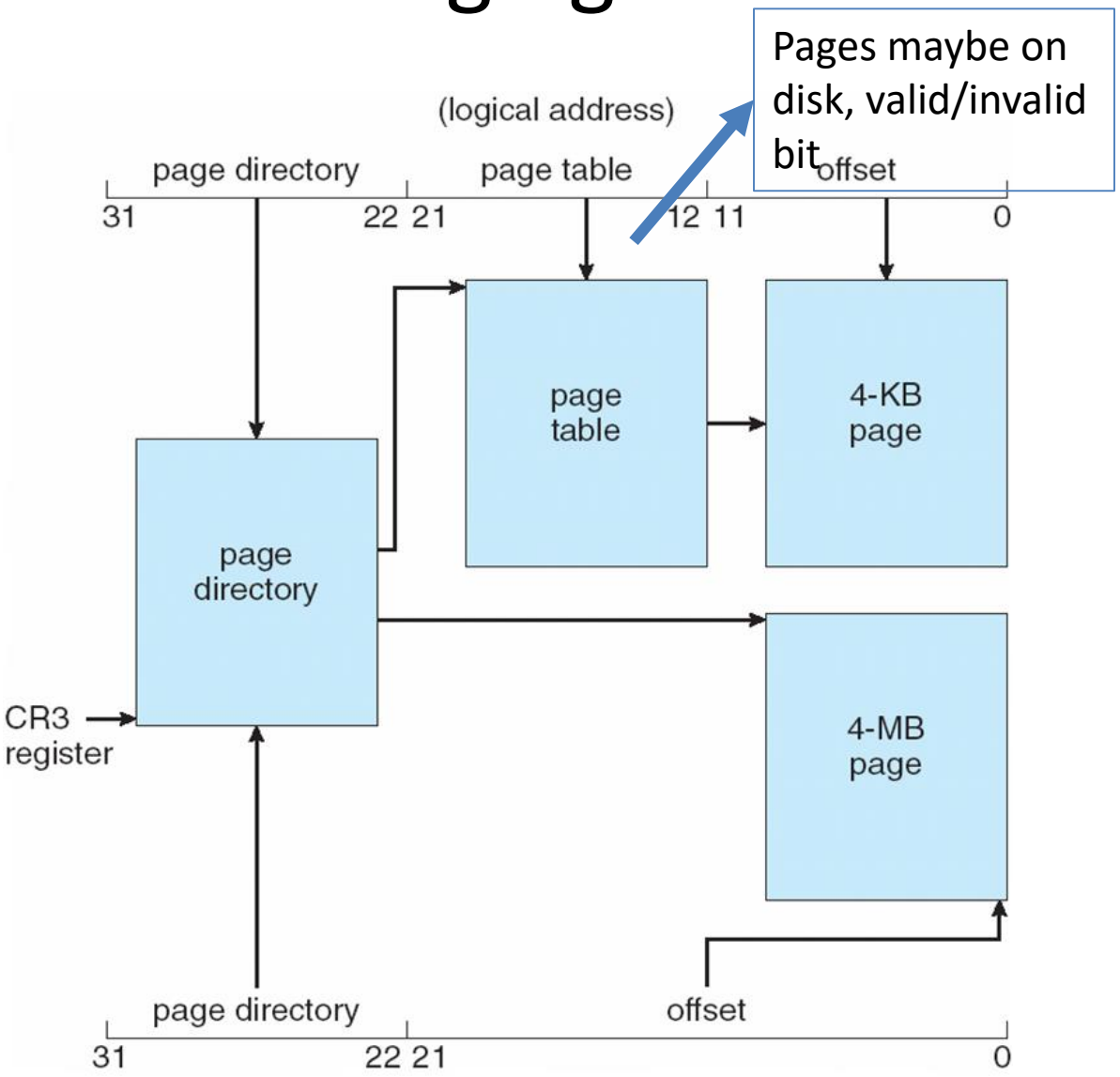
Example: The Intel Pentium



Intel Pentium Segmentation



Pentium Paging Architecture



Problem 1

Consider a system with byte-addressable memory, 32 bit logical addresses, 4 kilobyte page size and page table entries of 4 bytes each. Compute the size of the page table in the system in megabytes

Consider a machine with 64 MB physical memory and a 32-bit virtual address space. If the page size is 4 KB, what is the approximate size of the page table?

Problem 2

In a virtual memory system, size of virtual address is 32-bit, size of physical address is 30-bit, page size is 4 Kbyte and size of each page table entry is 32-bit. The main memory is byte addressable. Compute the maximum number of bits that can be used for storing protection and other information in each page table entry.

Problem 3

Consider a system with 2 level paging, where the page table is divided into 8k pages of size 16kB. The memory is byte addresable
The physical address space is 128MB which is divided into 4KB frames.
The page table entry of outer page table is 32 bits and page table entry size of inner page table is 64bits.
Show the logical and physical address format.
What will be the page table size of inner and outer page table?

Logical Address :

Outer Page (P_1) {13-bits}	Inner Page(P_2) {14-bits}	Page offset(d) {12-bits}

Physical Address :

Frame Number (f) {27-12=15-bits}	Frame offset(d) {12-bits}

Inner page table Size = $2^{P_2} *$ Inner page table entry size = 2^{14} * 64 bits = 2^{14} * 8 bytes = 128 KB.

Outer page table Size = $2^{P_1} *$ Outer page table entry size = 2^{13} * 32 bits = 2^{13} * 4 bytes = 32 KB

So, The inner Page table size is 128 KB and outer page table size is 32 KB.

Problem 4

Consider the segment table of a process

Segment No.	Base	Length
0	1219	700
1	2300	14
2	90	100
3	1327	580
4	1952	96

Process generates the following virtual addresses. Translate to physical address

1. 0, 430
2. 1, 11
3. 2, 100
4. 3, 425
5. 4, 95